

2. Runde

des

43. Bundeswettbewerbs Informatik

Aufgabe 1: Schmucknachrichten

Bearbeitet

von

Florian Bange
Teilnahme-ID: 74405

28. April 2025

Inhaltsverzeichnis

1	Einleitung	2
2	Mathematische Formulierung und Notation	3
3	Loesungsvorschlag fuer einheitliche Zeichenkosten	4
3.1	Huffmans Verfahren	4
3.2	Korrektheit von Huffmans Verfahren	6
3.3	Analyse der Platz- und Zeitkomplexitaet von Huffmans Verfahren	6
4	Approximationsverfahren fuer verschiedene Zeichenkosten	7
4.1	Approximationsverfahren I - Huffman-coding	7
4.2	Approximationsverfahren II - Laengenpraefixe	7
4.3	Approximationsverfahren III - Terminatorsymbol	8
5	Optimales Verfahren fuer verschiedene Zeichenkosten	9
5.1	Karps ILP-Formulierung des Problem	9
5.2	Generierung der Codewoerter	10
5.3	Analyse der Platz- und Zeitkomplexitaet	10
6	Implementierung	11
6.1	Umwandlung des Eingabeformats	11
6.2	Implementierungsdetails	11
7	Evaluierung anhand von Beispielen	12
7.1	Zusammenfassung der Ergebnisse	12
7.2	Beispielergebnisse der Approximationsverfahren	12
7.3	Optimale Loesungen fuer die Beispiele der BwInf-Website	16
7.4	Fazit der Evaluierung	32
8	Quellcode	33
8.1	Huffmans Verfahren	33
8.2	Gemeinsamer Quellcode	35
8.3	Approximationsverfahren I	36
8.4	Approximationsverfahren II	37
8.5	Approximationsverfahren III	40
8.6	ILP-Formulierung	42
9	Literatur	45

1. Einleitung

Dieses Dokument beinhaltet meine Dokumentation der ersten Aufgabe¹ der zweiten Runde des 43. Bundeswettbewerbs Informatik² in den Jahren 2024 und 2025.

Die vorliegende Problemstellung soll im Folgenden kurz mathematisch modelliert werden. Daraufhin werden die zwei Aufgabenteile der Aufgabenstellung separat gelöst. Zunaechst wird auf den ersten Aufgabenteil eingegangen, in welchem alle Perlen, bzw. Codezeichen dieselbe Laenge (bzw. Durchmesser oder Kosten) haben. Dieser wird mit Hilfe eines bekannten Verfahrens von Huffman aus 1952 gelöst, siehe [1]. Anschliessend wird der allgemeinere und deutlich komplexere zweite Aufgabenteil behandelt. In diesem haben die Codezeichen unterschiedliche Laengen. Fuer diesen werden zunaechst drei einfache Approximationsverfahren vorgestellt, welche eine sehr gute Laufzeit und mittelmässige Ergebnisse liefern. Daraufhin wird eine exakte Loesung, mit schlechterer Laufzeitkomplexitaet vorgestellt, welche den Integer-Linear-Programming Ansatz von Karp benutzt (siehe [3]) und erklart. Fuer beiden Verfahren wird anschliessend auch die Platz- und Laufzeitkomplexitaet betrachtet, wenngleich dies beim exakten Verfahren wegen der Verwendung eines ILP-Solvers nur oberflaechlich getan wird. Daraufhin wird die Implementierung der vorgestellten Loesungsverfahren erlaeutert. Am Ende folgt die Evaluierung der verschiedenen Loesungsansatze anhand der Beispiele der BwInf-Website. Dabei wird insbesondere die tatsaechliche Laufzeit und Gesamtlaenge der Ketten verglichen. Zuletzt werden die wichtigsten Teile des Quellcodes erlaeutert.

Genauer besteht die Loesung der ersten Teilaufgabe in Huffmans Verfahren, das einen praefixfreien Code generiert, indem iterativ ein sogenannter Huffman-Baum generiert wird. In der zweiten Teilaufgabe wird die allgemeine Problemstellung des praefixfreien Codes mit unterschiedlichen Zeichenkosten behandelt. Dazu werden drei Approximationsverfahren und ein exaktes Verfahren vorgestellt.

Alle drei Approximationsverfahren basieren auf der Idee auf irgendeine Art und Weise n Codewoerter zu konstruieren, die praefixfrei sind und dann dem haeufigsten Zeichen das Codewort mit geringsten Kosten, dem zweit-haeufigsten Zeichen das Codewort mit zweit-geringsten Kosten, usw. zu zuordnen. Das erste Approximationsverfahren basiert auf der einfachen Idee, die Loesung fuer einheitliche Codezeichenkosten, also Huffmans Verfahren zu verwenden, ohne dabei die Codezeichenkosten zu beachten. Das zweite Approximationsverfahren basiert auf der Idee aus [2], zunaechst die n kuerzesten Codewoerter zu konstruieren und dann jedes Codewort x_i durch ein Codewort $x'_i = \text{enc}(|x_i|)x_i$ zu ersetzen, wobei $\text{enc}(k)$ eine Zahl $k \in \mathbb{N}$ praefixfrei codiert. Dadurch entsteht ein praefixfreier Code mit relativ geringen Kosten. Das dritte Approximationsverfahren besteht in der Idee einen Terminalstring zu benutzen, der das Ende eines Wortes markiert. Konkret wird dieses Verfahren fuer den Fall eines Terminalstrings bestehend aus K mal s_1 vorgestellt. Dazu werden zunaechst die n kuerzesten Codewoerter generiert, welche nicht mehr als $K - 1$ mal s_1 aufeinanderfolgend enthalten, und welche nicht auf s_1 enden. Denn dann wird die Menge der Codewoerter, wobei jedem Wort der Terminalstring $t = s_1 \dots s_1$ angehaengt wird, praefixfrei.

Das optimale Loesungsverfahren besteht in einer Formulierung des Problems als ganzzahliges lineares Optimierungsproblem (engl. integer-linear-program, ILP), welches mit einer guten praktischen Laufzeit gelöst werden kann, aber theoretisch eine exponentielle Laufzeitkomplexitaet hat. Die Formulierung als ILP besteht aus einer Reihe von ganzzahligen Variablen, Linearen Ungleichungen zwischen diesen Variablen und einer zu minimierenden Funktion. Dieses Verfahren geht auf [3], Karp 1961 zurueck. Dieses ILP berechnet die optimale Anzahl an Codewoerter mit Kosten j fuer $j = 1, \dots, m$ fuer ein festes m . Daraus laesst sich dann eine Menge an Codewoertern generieren, und so vorgehen, wie bei den Approximationsverfahren.

¹Siehe https://bwinf.de/fileadmin/wettbewerbe/bundeswettbewerb/43/2_runde/Aufgaben432.pdf

²Siehe <https://bwinf.de/bundeswettbewerb/43/>

2. Mathematische Formulierung und Notation

Im Folgenden wird $\Sigma = \{a_1, a_2, \dots, a_n\}$ mit $n = |\Sigma|$ stets die Menge der Zeichen im Text, der codiert werden soll, bezeichnen. Dazu soll dann $P(x)$, mit $x \in \Sigma$, die Haeufigkeit des Zeichens x im vorgegeben Text sein. Insbesondere ist dann $\sum_{x \in \Sigma} P(x) = |\Sigma|$, wenn S der zu codierende Text ist. Weiter soll $C = \{s_1, s_2, \dots, s_r\}$ mit $r = |C|$ die Menge an Zeichen sein, die fuer die Codewoerter verwendet werden, die sogenannten Codezeichen. Dazu gibt $c(x) \in \mathbb{N}^+$ fuer $x \in C$ die Kosten des Zeichens x an.

Definition 1: Problem der Aufgabenstellung. Mit dieser Notation laesst sich das Problem der Aufgabenstellung (im allgemeinen Fall der Teilaufgabe b)) wie folgt definieren: Ein Alphabet Σ mit der Haeufigkeitsverteilung $P : \Sigma \rightarrow \mathbb{N}$, sowie eine Menge C der Codezeichen mit der Kostenfunktion $c : C \rightarrow \mathbb{N}$ ist vorgegeben. Nun sollen aus den Codezeichen ein praefixfreier Code bestehend aus n Codewoertern $X = (x_1, x_2, \dots, x_n)$ erstellt werden, sodass

$$\sum_{i=1}^n P(a_i) \cdot c(x_i)$$

minimiert wird. Dabei codiert x_i das Zeichen $a_i \in \Sigma$ und es gilt $x_i \in C^+$. Das heisst, x_i ist ein Wort, das mindestens ein Zeichen hat und dessen einzelne Zeichen aus C sind. Wir schreiben dann $x_i = s_{j_1} \dots s_{j_{l_i}}$, mit $l_i = |x_i|$. Dann wird $c(x_i)$ definiert als die Summe der Kosten $c(s_j)$ der einzelnen Zeichen des Codewortes x_i , also

$$c(x_i) = \sum_{j=1}^{l_i} c(s_j).$$

Weiter soll dieser Code praefixfrei sein. Das heisst, dass fuer jedes $x_i = s_{j_1} \dots s_{j_{l_i}}$ gilt, dass es kein anderes Codewort gibt das irgendeinem Praefix $s_{j_1} \dots s_{j_k}$ (mit $1 \leq k \leq l_i$) von x_i entspricht. Im Folgenden werden wir annehmen, dass $P(a_1) \geq P(a_2) \geq \dots \geq P(a_n)$ und $c(s_1) \leq c(s_2) \leq \dots \leq c(s_r)$ gilt.

Bemerkung 2: Man bemerke, dass

$$\sum_{i=1}^n P(a_i) \cdot c(x_i)$$

minimiert wird, genau dann, wenn

$$\frac{1}{|S|} \cdot \sum_{i=1}^n P(a_i) \cdot c(x_i) = \sum_{i=1}^n \frac{P(a_i)}{|S|} \cdot c(x_i)$$

minimiert wird. Deswegen kann man anstatt der Haeufigkeit $P(x)$ der einzelnen Buchstaben im vorgegeben Wort S auch die relative Haeufigkeit $p(x) = P(x)/|S|$ verwenden.

Bemerkung 3: Die erste Teilaufgabe entspricht nun dem Spezialfall, dass $c(s_i) = 1$ fuer alle $s_i \in C$ gilt. Dann sind fuer $x \in C^+$ die Kosten gegeben durch $c(x) = |x|$. Dann wird also die Summe

$$\sum_{i=1}^n P(a_i) \cdot c(x_i) = \sum_{i=1}^n P(a_i) \cdot |x_i|$$

minimiert, bzw. die Summe

$$\sum_{i=1}^n p(a_i) \cdot |x_i|.$$

3. Loesungsvorschlag fuer einheitliche Zeichenkosten

Wir haben nun die n Zeichen a_i mit relativen Haeufigkeiten $p(a_i)$, sowie r Codezeichen s_k vorgegeben. Damit wollen wir einen praefixfreien Code $X = (x_1, \dots, x_n)$ mit Codewoertern x_i aus Codezeichen konstruieren, sodass die gewichtete Summe der Codewortlaengen $\sum_{i=1}^n p(a_i) \cdot |x_i|$ minimiert wird. Dazu laesst sich Huffmans Verfahren aus 1952 verwenden, siehe [1]. Dieses Verfahren wird auch Huffman-Kodierung³ genannt.

3.1. Huffmans Verfahren

Huffmans Verfahren besteht darin, einen Baum zu konstruieren, der aehnlich zu einem Trie funktioniert. Hier hat jedes Blatt ein Zeichen a_i und jede Kante ein Codezeichen, fuer das sie steht. Geht man dann von der Wurzel zu einem Blatt, ohne dabei eine Kante doppelt zu benutzen, so erhaelt man das Codewort des Zeichens des Blatts. Weiter hat jeder Knoten (auch die Blaetter) eine Wahrscheinlichkeit, wobei die Wahrscheinlichkeit der Blaetter der Wahrscheinlichkeit der Zeichen a_i entspricht, und jeder Knoten als Wahrscheinlichkeit die Summe seiner Kinder hat, bzw. aquivalent die Summe der Wahrscheinlichkeiten aller Blaetter im Teilbaum des jeweiligen Knotens. Dieser Baum wird iterativ von den Blaettern zur Wurzel hin, von unten nach oben aufgebaut.

Wir wollen damit anfangen, die Struktur des Baumes durch eine Datenstruktur zu beschreiben, welche die Klasse *Node* benutzt. Objekte dieser Klasse sollen immer das Codezeichen, das sie beschreiben (im Falle eines Blattes), die Wahrscheinlichkeit (bzw. wegen der Ganzzahligkeit die absolute Haeufigkeit), und eine Liste an Kinderknoten beinhalten:

Algorithm 1: Klasse Node

```

1 character  $a$ 
2 integer  $p$ 
3 list of *Node  $childs(r)$ 
```

Das ***Node** soll hier andeuten, dass die Liste Pointer erhaelt, die auf die Kinderknoten zeigen, und insbesondere auch den Wert **NULL** haben koennen. Weiter soll $childs(r)$ bedeuten, dass die Anzahl der Kindknoten maximal r ist. Initial ist die Laenge der Lists also r und beinhaltet r mal **NULL**. Die Codezeichen, die zu jeder Kante gehoeren werden nun implizit durch die Position des Kindknotens in der $childs$ -Liste des Parentknotens beschrieben: Wenn ein Knoten v das Kind u hat und $v.childs[k] = u$ gilt, dann hat die Kante (v, u) das Codezeichen s_k . Dadurch wird auch sofort sichergestellt, dass alle Kanten, die von einem Knoten nach unten gehen, unterschiedliche Codezeichen haben.

Sobald dieser Baum mit Hilfe des Verfahrens, das gleich beschrieben wird, aufgebaut ist, kann man fuer jedes a_i einfach das dazugehoerige Codewort bestimmen, indem man eine rekursive Tiefensuche durchfuehrt:

Algorithm 2: Depth-first search

Input : Wurzel v des Baums

Output: Liste aller Zeichen a_i zusammen mit ihrem Codewort

```

1  $codes \leftarrow \emptyset$  // Liste der Zeichen und Codewoerter
2  $dfs(v, \varepsilon)$ 
3 Function  $dfs(u, w)$ :
4   if  $u$  is a Leaf then
5      $codes.add((u.a, w))$ 
6     return
7   end if
8   for  $i \leftarrow 0$  to  $r$  do
9      $dfs(u.childs[i], w + s_i)$ 
10  end for
11 return  $codes$ 
```

³Siehe auch https://en.wikipedia.org/wiki/Huffman_coding

Hier nimmt die rekursive Funktion *dfs* zwei Parameter an: Den aktuellen Knoten u und das aktuelle Codewort w , dass der Aneinanderreihung der Codezeichen der benutzten Kanten besteht. Diese Funktion prüft stets zuerst, ob es sich bei u um ein Blatt handelt (zum Beispiel, indem bei *Blaetter* die Länge von *childs* auf 0 gesetzt wird) fall dies der Fall ist, kann man das Zeichen $u.a$ mit dem Codewort w zur Liste *codes* der Codewörter hinzufügen. Anderenfalls wird rekursiv fuer jedes der r Kinder von u erneut die Funktion *dfs* aufgerufen. Nun mit dem neuen Knoten $u.childs[i]$ und $w + s_i$ als Uebergabeparameter, wobei $w + s_i$ die Konkatenation des Wortes w mit dem Zeichen s_i ist.

Zur Konstruktion des Baums wird das folgende iterative Verfahren benutzt:

1. **Initialisierung:** Erstelle fuer jedes der n Zeichen a_i ein Blattknoten u mit $u.a = a_i$, absoluter Haeufigkeit $u.p = P(a_i)$ und leerer Liste an Kindern. Lege alle Blattknoten in eine Prioritaetswarteschlange q ab, die so nach $u.p$ sortiert, dass die Knoten mit geringster absoluter Haeufigkeit p zuerst entfernt werden.
2. **Dummy-Knoten hinzufuegen (fuer $r > 2$):** Falls die Anzahl n der Blattknoten nicht die Bedingung $n - 1 \equiv 0 \pmod{r - 1}$ erfuehlt, werden so viele Dummy-Knoten mit Gewicht 0 und beliebigem Zeichen hinzugefueht, bis die Bedingung erfuehlt ist.
3. **Iteration:** Wiederhole, solange mehr als ein Knoten in der Warteschlange ist:
 - (a) Entferne die r Knoten mit den geringsten Gewichten aus der Warteschlange.
 - (b) Erstelle einen neuen Knoten u , wobei $u.p$ die Summe der absoluten Haeufigkeiten der r entnommenen Knoten ist.
 - (c) Setze die r entnommenen Knoten als Kinder des Knotens u .
 - (d) Fuege den neuen Knoten u wieder in die Prioritaetswarteschlange ein.
4. Der letzte verbleibende Knoten in der Warteschlange ist nun die Wurzel des Baumes.

Im Pseudocode mit Hilfe von obiger Datenstruktur sieht dies wie Folgt aus:

Algorithm 3: Huffman-coding

Input : Zeichen $a_1, \dots, a_n$, Haeufigkeiten $P(a_1), \dots, P(a_n)$, Anzahl der Codezeichen r

Output: Wurzel des Baums

```

1  $q \leftarrow \emptyset$  // Prioritaetswarteschlange
2 for  $i \leftarrow 0$  to  $n$  do
3    $l \leftarrow \text{Node}(a_i, P(a_i))$ 
4    $q.\text{push}(l)$ 
5 end for
6 while  $\text{NOT } q.\text{size}() - 1 \equiv 0 \pmod{r - 1}$  do
7    $l \leftarrow \text{Node}(\varepsilon, 0)$ 
8    $q.\text{push}(l)$ 
9 end while
10 while  $q.\text{size}() > 1$  do
11    $x \leftarrow \text{Node}(\varepsilon, 0)$ 
12   for  $i \leftarrow 0$  to  $r$  do
13      $c \leftarrow q.\text{pop}()$ 
14      $x.p = x.p + c.p$ 
15      $x.childs[i] = c$ 
16   end for
17    $q.\text{push}(x)$ 
18 end while
19  $x \leftarrow q.\text{pop}()$ 
20 return  $x$ 

```

Bemerke, dass es wichtig ist, dass Schritt 2 im allgemeinen Fall $r > 2$, sicherstellt, dass zu Beginn fuer die Anzahl n' der Blattnoten gilt, dass $n' - 1 \equiv 0 \pmod{r - 1}$. Dies stellt sicher, dass der resultierende Baum ein voller r -aerer Baum ist (das heisst, dass jeder Knoten, der kein Blatt ist genau r Kinder hat).

3.2. Korrektheit von Huffmans Verfahren

Das Ergebnis dieses Verfahrens ist ein praeifixfreier Code X , der die durchschnittliche Codewortlaenge $\sum_{i=1}^n p(a_i) |x_i|$ unter allen moeglichen praeifixfreien Codes fuer die gegebenen Zeichen und Codezeichen minimiert. Die Praefixfreiheit ergibt sich direkt aus der Baumstruktur, da kein Codewort (Pfad von der Wurzel zu einem Blatt) der Anfang eines anderen Codeworts sein kann. Die Minimierung der durchschnittlichen Codewortlaenge ist wegen des Greedy-Verfahrens sofort ersichtlich.

3.3. Analyse der Platz- und Zeitkomplexitaet von Huffmans Verfahren

Nun wollen wir kurz die Platz- und Zeitkomplexitaet von Huffmans Verfahren analysieren. Die Zeitkomplexitaet ist $\mathcal{O}((n+r) \log(n+r)) + \mathcal{O}(n+r) = \mathcal{O}((n+r) \log(n+r))$. Diese setzt sich zu sammen aus der Zeitkomplexitaet $\mathcal{O}((n+r) \log(n+r))$ des ersten Teils (Erstellung des Baumes) und der Zeitkomplexitaet $\mathcal{O}(n+r)$ zweiten Teils (Codewoerter aus dem Baum ablesen).

Der erste Teil (s. Algorithmus Huffman-coding) enthaelt zunaechst eine For-Schleife (Zeile 2 bis 5), die genau n Durchlaeufe hat, wobei jeder Durchlauf eine konstante Laufzeit hat. Darauf folgt eine While-Schleife die maximal r mal durchlaeuft (weil die Bedingung spaetestens nach $r-2$ Schritten erfuehlt sein muss). Auch hier hat jeder Schleifendurchlauf eine Konstante Laufzeit. Somit ergibt sich fuer die Zeilen 1 bis 9 zunaechst eine Laufzeit von $\mathcal{O}(n+r)$. Weiter wollen wir die uebrige While-Schleife betrachten. Hier muss man bemerken, dass die Warteschlange am Anfang genau $n' \leq n+r$ Knoten enthaelt. Jeder Durchlauf der While-Schleife hat eine Laufzeit von $\mathcal{O}(\log(n+r) + r)$, weil hier eine Operation mit konstanter Laufzeit (Zeile 11), eine Operation mit Laufzeit $\mathcal{O}(\log(n+r))$ (Zeile 17), und eine Schleife TODO. Die While-Schleife wird also maximal $\frac{n+r}{r-1}$ durchlaufen.

Fuer die Laufzeit des zweiten Teils muessen wir die Anzahl der Knoten des Baumes betrachten. Denn die Tiefensuche besucht jeden Knoten des Baumes genau einmal und fuehrt fuer jeden Knoten konstant viele Operationen durch. Nun entspricht die Anzahl der Knoten genau der Summe aus den Blaetter $n' \leq n+r$ und de Anzahl der Durchlaeufe der While-Schleife. Also hoechstens $n+r + \frac{n+r}{r-1} \leq 2n+r = \mathcal{O}(n+r)$.

Bemerkt man nun, dass in der Praxis stets $r \in \mathcal{O}(n)$ gilt, weil fuer $r \geq n$ sofort jedes Zeichen a_i einem Codezeichen s_k zugeordnet werden koennte, so ergibt sich fuer Teil 1 die Laufzeit $\mathcal{O}(n \log n)$ und fuer Teil 2 die Laufzeit $\mathcal{O}(n)$, also insgesamt $\mathcal{O}(n \log n)$.

Die Platzkomplexitaet laesst sich erneut durch Betrachtung der beiden einzelnen Teile des Verfahrens bestimmen. Dazu bemerkt man, dass der Platzverbrauch im ersten Teil nur von den Eingaben, der Warteschlange und dem Baum abhaengt. Erstere haben Platzkomplexitaet $\mathcal{O}(n)$, waehrend die Warteschlange Platzkomplexitaet $\mathcal{O}(n+r)$ hat, da anfang maximal $n+r$ Elemente zur Warteschlange hinzugefuegt werden, und niemals mehr hinzu kommen. Der Baum wiederum hat wie zuvor bemerkt $\mathcal{O}(n+r)$ viele Knoten. Damit laesst sich insgesamt die Platzkomplexitaet $\mathcal{O}(n+r)$ bestimmen. Im zweiten Teil kommt kein direkter Platzverbrauch hinzu, aber es ist zu bemerken, dass wegen der rekursiven Struktur der Funktion fuer maximal so viele Ruecksprungsadresse gespeichert werden muessen, wie der Baum tief ist. In diesem Fall laesst sich die Tiefe einfach durch die Anzahl der Knoten abschaetzen, also $\mathcal{O}(n+r)$. Insgesamt ergibt sich also die Platzkomplexitaet $\mathcal{O}(n+r)$. Mit obiger Bemerkung erhaelt man Die Platzkomplexitaet $\mathcal{O}(n+r) = \mathcal{O}(n)$.

4. Approximationsverfahren fuer verschiedene Zeichenkosten

Nachdem der Fall mit einheitlichen Zeichenkosten effizient geloest wurde, stellt sich die Frage, ob es fuer den Fall mit verschiedenen Zeichenkosten auch ein optimales und effizientes Verfahren gibt. In der uns bekannten Literatur scheint dies nicht der Fall zu sein, wenngleich das Problem nicht als NP-schwer bekannt ist. Wegen des Fehlens von polynomiellen Algorithmen des weitreichend untersuchten Problems, gehen wir davon aus, dass es sich um ein NP-schweres Problem handelt, welches *vermutlich* nicht effizient geloest werden kann. Aus demselben Grund wollen wir nun zunaechst mehrere Approximationsverfahren angeben, die das Problem fuer verschiedene Zeichenkosten nicht notwendigerweise optimal, aber da fuer in deutlich schnellerer Zeit loesen als das optimale Verfahren, welches im Folgenden Kapitel behandelt wird.

Hier werden insgesamt drei Approximationsverfahren vorgestellt. Das erste behandelt die naheliegende Idee, das Verfahren fuer einheitliche Zeichenkosten auch hier zu benutzen. Das zweite Verfahren basiert auf der Hauptidee aus [2], wo zunaechst ein nicht-praefixfreier Code generiert wird, welcher anschliessend praefixfrei gemacht wird, indem jedes Codewort als Praefix eine Codierung seiner Laenge erhaelt: $x'_i = \text{enc}(|x_i|)x_i$. Das dritte Approximationsverfahren basiert ebenfalls darauf, einen nicht-praefixfreien Code praefixfrei zu machen. Dazu wird hier ein String aus Codezeichen $t = s_1 \dots s_1$ (K mal) gewaehlt, der das Ende von einem Codewort markieren soll. Damit dieses Verfahren funktioniert darf keines der anfangs gegebenen Woertern den String t enthaelten, oder auf s_1 enden. Alle hier behandelten Verfahren werden am Ende der Dokumentation mit Hilfe der Beispiele der BwInf-Website verglichen.

4.1. Approximationsverfahren I - Huffman-coding

Zunaechst wollen wir kurz das naheliegende Verfahren erwaehnen, das zuvor beschriebene Huffman-Coding auch im Fall der unterschiedlichen Zeichenkosten zu verwenden. Somit ignoriert man effektiv die Zeichenkosten der Codezeichen und erreicht offensichtlich (fast) nie die optimale Loesung. Nachdem die Codewoerter mit Huffmans Verfahren generiert wurden, ist es sinnvoll diese absteigend nach Kosten zu sortieren. Denn wenn die Zeichen aufsteigend nach Haeufigkeit sortiert sind, ist es fuer die gegebenen Codewoerter offensichtlich optimal das i -te Zeichen mit dem i -ten Codewort zu codieren.

4.2. Approximationsverfahren II - Laengenpraefixe

Das Approximationsverfahren, das nun vorgestellt wird, basiert auf der Hauptidee aus [2]⁴. Dort werden zunaechst n Codewoerter mit den n minimalsten Kosten unter allen moeglichen Codewoertern erstellt. Diese seien $X = (x_1, \dots, x_n)$ mit $c(x_1) \leq \dots \leq c(x_n)$. Natuerlich sind diese nicht notwendigerweise (bzw. in den meisten Faellen) nicht praefixfrei, aber es handelt sich um den Code, der offensichtlich

$$\sum_{i=1}^n P(a_i) \cdot c(x_i)$$

minimiert, wenn $P(a_1) \geq \dots \geq P(a_n)$ gilt. Ausgehend von diesem Code soll nun ein praefixfreier Code $X' = (x'_1, \dots, x'_n)$ erstellt werden, sodass die Kosten $c(x'_i)$ nur etwas groeszer sind als die Kosten $c(x_i)$. Dazu waehlt man $x'_i = \text{enc}(|x_i|)x_i$. Dabei ist $|x_i|$ die Laenge von x_i (also die Anzahl der Codezeichen in x_i) und $\text{enc}(|x_i|)$ eine praefixfreie Codierung von $|x_i|$. Genauer ist enc eine Abbildung von $\mathbb{N}$ zu Strings aus Codezeichen mit Laenge ≥ 1 . Dabei soll enc injektiv und praefixfrei sein. Letzteres bedeutet, dass kein $\text{enc}(k)$ echter Praefix eines andere $\text{enc}(l)$ ist. Zunaechst zeigen wir, dass dieser Code tatsaechlich praefixfrei ist:

Lemma 4: X' ist praefixfrei. Fuer einen Widerspruch nehmen wir an, dass das Codewort x'_i ein echter Praefix des Codeworts x'_j ist. Dann ist also $x'_i = \text{enc}(|x_i|)x_i$, $x'_j = \text{enc}(|x_j|)x_j$ und $x'_j = x'_i\beta$ fuer einen String β aus Codezeichen mit $|\beta| \geq 1$. Also gilt

$$\text{enc}(|x_j|)x_j = \text{enc}(|x_i|)x_i\beta.$$

Daraus folgt bereits $\text{enc}(|x_j|) = \text{enc}(|x_i|)$. Denn waeren $\text{enc}(|x_j|)$ und $\text{enc}(|x_i|)$ unterschiedlich lang, so waere einer der Strings ein echter Praefix des anderen und $\text{enc}(k)$ nicht praefixfrei. Sind sie aber gleich lang, so muessen

⁴Dabei moechte ich erwaehnen, dass ich nur das erste Kapitel des Papers gelesen habe und die Details selbst entwickelt wurden

sie bereits gleich sein. Nun hat man also $x_j = x_i\beta$, sodass $|x_j| > |x_i|$, aber aus $\text{enc}(|x_j|) = \text{enc}(|x_i|)$ folgt bereits $|x_j| = |x_i|$. Somit besteht ein Widerspruch!

Dies passt zu der Intuition, dass man einen Text, der mit diesem Code codiert ist eindeutig decodieren kann, indem man stets zunachst die Laenge des darauffolgenden Codeworts einliest, und dann das Codewort eindeutig identifizieren kann, weil man die Laenge bereits kennt. Dies setzt allerdings voraus, dass man das Ende der Laengenangabe eindeutig erkennen kann, beispielsweise durch ein eindeutigen String aus Codezeichen, dass in der codierung einer Laenge nicht vorkommt. Diese Idee wird nun benutzt, um die oben beschriebene Funktion enc zu definieren:

Definition 5: Die Abbildung $\text{enc} : \mathbb{N} \rightarrow C^+$. Wir wollen hier ein einfaches Verfahren beschreiben, um beliebige Laengen $|x_i| \geq 1$ mit Codezeichen zu codieren, sodass jedem $k \in \mathbb{N}$ ein eindeutiger String in C^+ zugeordnet wird (also sodass enc injektiv ist), und kein $\text{enc}(k)$ Praefix eines anderen $\text{enc}(l)$ ist. Ausserdem wollen wir, dass die entstehenden Codierungen moeglichst geringe Kosten haben. Eine einfache (und nicht notwendigerweise optimale) Art das zu erreichen, ist es die starndartmaessige Binaerdarstellung der Zahl k zu verwenden, wobei anfaengliche Nullen weggelassen werden, aber statt 0 und 1 die Strings s_1s_2 und s_2s_1 zu verwenden. Um dies nun praefixfrei zu machen fuegt man an das Ende noch s_1s_1 an.

Nun bleibt zu erklaren, wie man den anfaenglichen Code X konstruiert. Dazu konstruieren wir einen Baum, dessen Knoten fuer Codezeichen stehen, sodass jeder Knoten am ende dem Wort entspraechen wird, welches entsteht wenn man von der Wurzel zu diesem Knoten geht und die Codezeichen jedes Knotens, den man besucht hintereinander schreibt. Dabei ist die Wurzel insbesondere das leere Wort.

Um diesen Baum nun zu konstruieren, legen wir eine Prioritaetswarteschlange an, welche die naechsten hinzuzufuegenden Knoten beinhalten wird. Solange der Baum weniger als n Knoten hat, wird jeweils der Knoten mit minimalem Kosten aus der Warteschlange entfernt. Zu Beginn befindet sich nur das erste Kind der Wurzel in der Warteschlange. Wird ein Knoten aus der Warteschlange genommen wird dieser zum Baum hinzugefuegt. Fall dessen Parent-Knoten weniger als r Kinder hat wird das naechste Kind des Parent-Knotens in die Warteschlange hinzugefuegt. Ebenfalls wird das erste Kind des aktuellen Knotens in die Warteschlange hinzugefuegt. Die Kosten eines Knotens in der Warteschlange entspraechen immer den Kosten, die das zum Knoten gehoerige Codewort haette.

4.3. Approximationsverfahren III - Terminatorsymbol

Wir waehlen $K \geq 1$ beliebig und setzen $t = s_1 \dots s_1$ (K mal). Der String t ist nun unser Terminatorsymbol. Dieses soll verwendet werden, um das Ende eines Codewords zu markieren. Damit dies eindeutig ist generieren wir zunachst, wie zuvor, die n kuerzesten Worte aus C^* (gewichtet nach Kosten). Aber dieses mal sollen diese Worte zusaetzlich die Eigenschaft haben, dass keines der Worte t enthaelt, oder auf s_1 endet. Diese Worte seien $X = (x_1, \dots, x_n)$. Wir setzen nun $X' = (x'_1, \dots, x'_n)$ mit $x'_i = x_i t$. Zunachst zeigen wir, dass dieser Code tatsaechlich praefixfrei ist:

Lemma 6: X' ist praefixfrei. Angenommen, es waere $x_i t$ ein echter Praefix von $x_j t$. Aber dann ist $x_i t$ mindestens um eins kuerzer als $x_j t$. Also muss t entweder vollstaendig in x_j liegen, oder x_j endet auf 0. Beides gibt einen Widerspruch!

Wieder bleibt zu zeigen, wie man die Worte X am Anfang konstruiert. Wir bei Approximationsverfahren II wollen wir greedy einen Baum konstruieren, der uns am Ende die Woerter gibt. Dieses mal muessen wir allerdings aufpassen, dass die zwei Bedingungen beachtet werden. Dies ist moeglich, indem man jedes mal wenn man ein K -tes s_1 zum Baum hinzufuegen wuerde, einen leeren Knoten (bzw. extra markierten) Knoten zum Baum hinzufuegt, der keine Kinder mehr haben kann. Ausserdem zaehlt man alle Knoten die fuer s_1 stehen nicht mit und fuegt ihre Codewoerter am Ende nicht hinzu.

5. Optimales Verfahren fuer verschiedene Zeichenkosten

Nun soll das optimale Verfahren fuer verschiedene Zeichenkosten vorgestellt werden. Dieses benutzt das Verfahren aus [3] von Karp. Im Folgenden sei abweichend von der vorherigen Notation X_i das i . Symbol (also das was vorher a_i war), weil a_j anders benutzt wird.

5.1. Karps ILP-Formulierung des Problem

Karp zeigt in [3], dass ein praeifixfrei Code mit $a_j \geq 0$ Codewoertern mit Kosten $j \geq 1$ genau dann existiert, wenn $b_j \geq 0$ existieren mit

$$a_j + b_j \leq \sum_{k=1}^r b_{j-c_k}$$

fuer alle $j > 0$, wobei $b_0 = 1$, und $b_j = 0$ fuer $j < 0$.

Zunaechst wollen wir ein hinreichend groszes $m \in \mathbb{N}$ waehlen, sodass $1 \leq j \leq m$ gilt. Es gibt also maximal Codewortkosten von m . Daraus laesst sich ein Optimierungsproblem formulieren. Dazu bemerkt man, dass wir

$$\sum_{i=1}^n P(X_i) \cdot l_i$$

minimieren wollen, wobei l_i die Kosten des Codeworts fuer x_i sind. Um dies modellieren zu koennen, fuehrt Karp Entscheidungsvariablen y_{ij} ein:

$$y_{ij} = \begin{cases} 1, & \text{wenn das Symbol } x_i \text{ ein Codewort mit Kosten } j \text{ erhaelt } (l_i = j) \\ 0, & \text{sonst} \end{cases}$$

Daraus ergibt sich sofort

$$l_i = \sum_{j=1}^m j y_{ij}, \text{ und } a_j = \sum_{i=1}^n y_{ij}.$$

Wir koennen nun also

$$\sum_{i=1}^n P(X_i) \cdot l_i = \sum_{i=1}^n P(X_i) \cdot \left(\sum_{j=1}^m j y_{ij} \right) = \sum_{i=1}^n \sum_{j=1}^m P(X_i) j \cdot y_{ij}$$

minimieren. Wir muessen allerdings die Gleichungen

$$\sum_{j=1}^m y_{ij} = 1$$

fuer $i = 1, \dots, n$ hinzufuegen, um sicherzustellen, dass jedes Zeichen eindeutige Kosten hat. Die urspruengliche Ungleichung sieht nun wie folgt aus:

$$\sum_{i=1}^n y_{ij} + b_j \leq \sum_{k=1}^r b_{j-c_k}$$

fuer alle $1 \leq j \leq m$, wobei immer noch $b_0 = 1$, und $b_j = 0$ fuer $j < 0$.

Die ILP-Formulierung

Die finale ILP-Formulierung sieht also wie folgt aus:

Variablen:

Binaere Entscheidungsvariablen y_{ij} fuer $i = 1, \dots, n$ und $j = 1, \dots, m$; sowie Ganzzahlen $b_j \geq 0$ fuer $j = 1, \dots, m$.

Nebenbedingungen:

$$\sum_{i=1}^n y_{ij} + b_j \leq \sum_{k=1}^r b_{j-c_k}, \text{ und } \sum_{j=1}^m y_{ij} = 1$$

Minimiere

$$\sum_{i=1}^n \sum_{j=1}^m P(X_i) j \cdot y_{ij}$$

5.2. Generierung der Codewörter

Nun haben wir also die $a_1, \dots, a_m$ gegeben. Daraus lassen sich n Codewörter generieren, sodass es genau a_j Codewörter mit Kosten j gibt. Dafür lässt sich dieses iterative Verfahren verwenden.

Algorithm 4: Generate code words

Input : Werte $a_1, \dots, a_m$
Output: Liste an Codewörtern

```

1  $q \leftarrow \emptyset$  // Warteschlange
2  $codes \leftarrow \emptyset$ 
3 while NOT alle  $a_j$  gleich 0 do
4    $w \leftarrow q.pop()$ 
5    $c \leftarrow$  Kosten von  $w$  // Summe der Kosten der Codezeichen in  $w$ 
6   if  $a_c > 0$  then
7     //  $a_c = 0$  fuer  $c < 1$  oder  $c > m$ 
8      $codes.add(w)$ 
9      $a_c \leftarrow a_c - 1$ 
10  end if
11  else
12    for  $k = 1$  to  $r$  do
13       $q.add(w + s_k)$ 
14    end for
15  end if
16 end while
17 return  $codes$ 
```

5.3. Analyse der Platz- und Zeitkomplexität

Eine konkrete Analyse der Platz- und Zeitkomplexität ist schwer, weil man dafür den konkreten ILP-Solver mit einbeziehen müsste. Wir beschränken uns hier darauf, die Anzahl an Variablen und Nebenbedingungen zu analysieren. Weiter sei gesagt, dass ein ILP-Solver zwar theoretisch immer exponentielle Laufzeit hat, aber darauf ausgelegt ist, in der Praxis eine gute Laufzeit zu erreichen.

Die Anzahl der Variablen ist offensichtlich $n \cdot m + m = m(n + 1) = \mathcal{O}(nm)$. Weiter ist die Anzahl der Nebenbedingungen offensichtlich $m + n$. Weiter ist die Wahl der Laufzeit stark abhängig von der Wahl von m . Diese Zahl sollte hinreichend hoch gewählt werden, oder solange iterativ erhöht werden, bis keine Verbesserung mehr erzielt wird.

6. Implementierung

Nun soll auf die konkrete Implementierung der vorgestellten Lösungsverfahren eingegangen werden. Dabei wird zunächst behandelt, wie aus dem vorgegebenen Eingabeformat die eigentlich erwarteten Eingabedaten erhalten werden. Denn die vorgegebene enthält nur einen Text der codiert werden soll und die Gewichte der Codezeichen, wie benötigen aber die einzelnen Zeichen und ihre Häufigkeit, sowie die genauen Codezeichen mit ihren Gewichten. Ausserdem werden die genauen Implementierungsdetails erklärt.

6.1. Umwandlung des Eingabeformats

Das vorgegebene Eingabeformat enthält die Kosten der Codezeichen, sowie einen Text der codiert werden soll. Somit kann man diesen Text einfach Zeichenweise durchgehen, und für jedes Zeichen in einer map speichern, wie oft dieses Zeichen vorkommt. Diese map wird dann in ein vector aus Paaren jeweils bestehend aus Häufigkeit des Zeichens und dem Zeichenselbst umgewandelt.

6.2. Implementierungsdetails

Alle Lösungsverfahren wurden in C++ implementiert. Dabei besteht die erste Hürde darin, die nichtstandard Zeichen ordentlich zu speichern und wieder auszugeben. Dazu wurden die Datentypen *u32string* sowie *char32_t* verwendet. Ausserdem lässt sich zur Umwandlung die Funktion `std::wstring_convert<std::codecvt_utf8<char32_t>, char32_t>` verwenden.

Ein weiteres wichtiges Detail der Implementierung besteht darin, dass wir die Codezeichen nicht vorgegeben bekommen, deswegen werden die Codewörter an einigen Stellen durch Listen von Codezeichenidezen dargestellt. Diese können dann elementweise in ein Codewort umgewandelt werden. Wobei wir die ersten 10 Codezeichen als 0 bis 9 definieren, und die weiteren 26 als A bis Z.

Weiter werden in mehreren der Approximationsverfahren Bäume benutzt, welche durch Knoten mit Verweisen auf Kinder durch Pointer implementiert wurden. Dabei ist es sinnvoll für die Pointer `std::shared_ptr<Node>` zu verwenden, um mögliche Fehler mit Pointern zu vermeiden.

In den Approximationsverfahren und auch in dem exakten Verfahren werden sehr ähnliche Vorgehensweisen benutzt. Genauer wird jedes mal zunächst mal die Eingabe eingelesen und passend umgewandelt, um anschliessend eine Liste an Codewörtern zu generieren, welche anschliessend immer nach Kosten sortiert werden, und dann in logischer Reihenfolge zu den Zeichen zugeordnet werden. Deswegen lässt sich ein grosser, gemeinsamer Teil der Verfahren auslagern, in eine Datei, die wir *common.h*, bzw. *common.cpp* genannt haben.

Zuletzt wollen wir auf die ILP-Lösung eingehen. Hier wurde eine Bibliothek von Google benutzt, genauer OR-Tools. Hier wurde konkret der SCIP Solver benutzt.

7. Evaluierung anhand von Beispielen

7.1. Zusammenfassung der Ergebnisse

Nun wollen wir die vorgestellten Lösungsverfahren mit Hilfe der vorgegeben Beispieleingaben evaluieren. Dazu erwähnen wir zunächst, dass in Approximation III der Wert $K = 2$ gewählt wurde und im exakten ILP-Verfahren der Wert $m = 200$. Nun wollen wir zunächst eine Übersicht über die Beispieleingaben geben:

schmuck[...]	0	00	01	1	2	3	4	5	6	7	8	9
Anzahl der Zeichen	12	28	45	25	9	9	14	40	34	82	321	674
Anzahl der Codezeichen	2	3	5	3	2	3	2	7	3	10	5	4

Diese Tabelle führt für jede der 12 Beispieleingaben auf, wie viele verschiedene Zeichen der zu codierende Text hat, und wie viele Codezeichen es gibt. Die folgende Tabellen sollen wiederum die Ergebnisse der Lösungsverfahren zusammenfassen. Dazu wird für jeden Algorithmus und jede Eingabe angegeben, welche Laufzeit (in Sekunden) und welche Gesamtkosten der jeweilige Algorithmus für die jeweilige Eingabe erreicht.

Eingabe	Huffman-coding	
	Gesamtkosten	Laufzeit
schmuck0	113	0.000
schmuck00	372	0.000
schmuck01	1150	0.000

Tabelle 1: Ergebnisse der Lösung für einheitliche Codezeichenkosten

Eingabe	Approximation I		Approximation II		Approximation III	
	Gesamtkosten	Laufzeit	Gesamtkosten	Laufzeit	Gesamtkosten	Laufzeit
schmuck0	113	0,000	215	0,000	140	0,000
schmuck00	372	0,000	873	0,001	520	0,001
schmuck01	1150	0,000	3405	0,001	1953	0,001
schmuck1	199	0,000	382	0,001	247	0,000
schmuck2	213	0,000	210	0,000	168	0,000
schmuck3	324	0,000	589	0,000	422	0,000
schmuck4	170	0,000	262	0,000	197	0,000
schmuck5	3865	0,000	6770	0,001	4406	0,000
schmuck6	264	0,000	462	0,000	296	0,000
schmuck7	153285	0,003	455197	0,024	268674	0,025
schmuck8	3558	0,001	6365	0,004	3997	0,007
schmuck9	39145	0,002	63413	0,008	41227	0,029

Tabelle 2: Ergebnisse der Approximationsverfahren für unterschiedliche Codezeichenkosten

In der folgenden Tabelle ist weiter die Nähe der Approximationsverfahren zum optimalen Ergebnis für jede Eingabedatei in Prozent angegeben.

7.2. Beispielergebnisse der Approximationsverfahren

Bevor wir die optimalen Ergebnisse vorstellen, sollen nun kurz die Ergebnisse der Approximationsverfahren an ausgewählten Beispielen vorgestellt werden. Konkret geben wir die Ergebnisse der Approximationsverfahren für die Eingaben schmuck0, schmuck1 und schmuck2 an. Für alle weiteren Ergebnisse verweisen wir auf die Ausgabedateien.

Eingabe schmuck0

Die Ergebnisse der drei Approximationsverfahren für die Eingabe schmuck0:

Von links nach rechts: Approximation I, Approximation II und Approximation III.

Codewort von Approximation I: (Kosten 113)

Eingabe	ILP	
	Gesamtkosten	Laufzeit
schmuck0	113	0,332
schmuck00	372	0,616
schmuck01	1150	0,934
schmuck1	191	0,135
schmuck2	135	0,062
schmuck3	279	0,075
schmuck4	137	0,116
schmuck5	3162	0,388
schmuck6	234	0,329
schmuck7	134559	0,865
schmuck8	3287	2,973
schmuck9	36597	24,291

Tabelle 3: Ergebnisse des optimalen Verfahrens fuer unterschiedliche Codezeichenkosten

Eingabe	Approximation I	Approximation II	Approximation III
schmuck0	0,00	90,267	23,89
schmuck00	0,00	134,68	39,78
schmuck01	0,00	196,09	69,83
schmuck1	4,18	100,00	29,32
schmuck2	57,78	55,56	24,44
schmuck3	16,13	111,11	51,25
schmuck4	24,09	91,24	43,80
schmuck5	22,23	114,10	39,34
schmuck6	12,82	97,44	26,50
schmuck7	13,92	238,29	99,67
schmuck8	8,24	93,64	21,60
schmuck9	6,96	73,27	12,65

Tabelle 4: Relativer Fehler der Approximationsverfahren zum optimalen Ergebnis in Prozent

Symbol	Code	Kosten
E	000	3
	001	3
N	011	3
I	110	3
S	111	3
R	0101	4
O	1000	4
M	1001	4
L	1010	4
D	1011	4
H	01000	5
C	01001	5

Symbol	Code	Kosten
E	00	2
	10000	5
N	10001	5
I	10010000	8
S	10010001	8
R	10010010	8
O	10010011	8
M	101000000	9
L	101000001	9
D	101000010	9
H	101000011	9
C	101000100	9

Symbol	Code	Kosten
E	00	2
	100	3
N	0100	4
I	1100	4
S	01100	5
R	10100	5
O	11100	5
M	010100	6
L	011100	6
D	101100	6
H	110100	6
C	111100	6

10111100000011111000011011000001111100010101010001101111001010011101001100100001
010011110100101000000110011000011

Codewort von Approximation II: (Kosten 215)

10100001010010000001000010010001100100111000110001001000010010001100100111010000
011010000011000010100001010010000100100101000010010000101000000010100000000100100
1010000100100011010001001010000110010010000100010010001

Codewort von Approximation III: (Kosten 140)

10110011000010001100111000100010000100011001110001110001110010010110011001010010
011000101000101000010100100011001111001101000011000100000100

Eingabe schmuck1

Die Ergebnisse der drei Approximationsverfahren fuer die Eingabe schmuck1:

Von links nach rechts: Approximation I, Approximation II und Approximation III.

Symbol	Code	Kosten
e	01	2
t	11	2
	000	3
w	001	3
r	02	3
n	100	3
i	101	3
s	21	3
f	002	4
b	102	4
I	120	4
B	121	4
"	200	4
ü	201	4
u	122	5
o	2020	6
m	2021	6
k	2200	6
h	2201	6
d	2210	6
a	2211	6
W	222	6
N	2022	7
F	2202	7
D	2212	7

Symbol	Code	Kosten
e	00	2
t	10000	5
	10001	5
w	10002	6
r	10010000	8
n	10010001	8
i	10010010	8
s	10010011	8
f	10010002	9
b	10010012	9
I	10010020	9
B	10010021	9
"	101000000	9
ü	101000001	9
u	101000010	9
o	101000011	9
m	101000100	9
k	101000101	9
h	101000110	9
d	101000111	9
a	10010022	10
W	101000002	10
N	101000012	10
F	101000102	10
D	101000112	10

Symbol	Code	Kosten
e	00	2
t	100	3
	0100	4
w	1100	4
r	200	4
n	01100	5
i	0200	5
s	10100	5
f	11100	5
b	1200	5
I	2100	5
B	010100	6
"	011100	6
ü	01200	6
u	02100	6
o	101100	6
m	10200	6
k	110100	6
h	111100	6
d	11200	6
a	12100	6
W	20100	6
N	21100	6
F	2200	6
D	02200	7

Codewort von Approximation I: (Kosten 199)

12122212020222202000211101220111000002201020002002212101010001211221002210012100
1011011101100000120100002202002202122111110122000010111110201001010210201200

Codewort von Approximation II: (Kosten 382)

10010021101000002100100201010000121010001021000110010011100000010100011010000100
01100100021010000011001000010001101000000101000112100100100010001100100211010000
10100100011010001110010010011100020010010010100000010010001100011001002010010001
10010002101000011100100001010001001001002210000100100101010001011000200100001000
010010012001000200100100001001001200101000000

Codewort von Approximation III: (Kosten 247)

01010020100210021100220001001010010000111100100010011100012002000100011100022000
20000010001010002100011001120000101001100000200100000110001002100011001110010110
020010200121001000200110100110000100100120000110000200120000011100

Die Ergebnisse der drei Approximationsverfahren fuer die Eingabe schmuck2:

Symbol	Code	Kosten
a	0000	4
h	1	5
g	0001	8
f	0010	8
e	0100	8
d	0011	12
c	0101	12
b	0110	12
	0111	16

Symbol	Code	Kosten
a	00	2
h	10000	9
g	10001	13
f	10010000	16
e	101000000	17
d	10010001	20
c	10010010	20
b	100101000000	24
	1001100000000	25

Symbol	Code	Kosten
a	00	2
h	100	7
g	0100	8
f	1100	12
e	01100	13
d	10100	13
c	010100	14
b	11100	17
	011100	18

[illegible][illegible][illegible]

7.3. Optimale Loesungen fuer die Beispiele der BwInf-Website

Nun wollen wir fuer jede der Beispieleingaben eine optimale Loesung angeben. Dabei wird jeweils zuneachst eine Tabelle der Codezeichen zusammen mit ihren Kosten, sowie eine Tabelle mit allen Zeichen des vorgegebenen Textes zusammen mit ihrer Haeufigkeit im Text, sowie ihrem Codewort und dessen Kosten angegeben. Dabei wurde diese Tabelle bei den Eingabedateien schmuck8 und schmuck9 weggelassen, da sie relativ lang sind. Die Codetabellen sind aber in den Loesungsdateien zu finden. Daraufhin folgt jeweils der codierte Text (auszer bei der Eingabedatei schmuck7, da dieser sehr lang ist). Auszerdem moechten wir darauf hinweisen, dass in einigen Codetabellen die konkreten Symbole nicht (richtig) angezeigt werden, da Latex sie nicht korrekt verarbeiten kann. In den Ausgabedateien sind die aber vollstaendig angegeben.

1. Beispiel: schmuck0.txt, $n = 12$

Codezeichen	0	1
Kosten	1	1

Tabelle 5: Codezeichen fuer schmuck0.txt

Symbol	Haeufigkeit	Code	Laenge / Kosten
E	5	000	3
	5	001	3
N	4	010	3
I	4	011	3
S	3	100	3
R	2	1010	4
O	2	1011	4
M	2	1100	4
L	2	1101	4
D	2	1110	4
H	1	11110	5
C	1	11111	5

Tabelle 6: Codetabelle fuer schmuck0.txt

Codierter Text:

```
11100110000011001011010010000001100101111011101001111001110100010111100110000010
100011001111111110000011010000010
```

Kosten: 113

2. Beispiel: schmuck00.txt, $n = 28$

Codezeichen	0	1	2
Kosten	1	1	1

Tabelle 7: Codezeichen fuer schmuck00.txt

Codierter Text:

```
22211200021122022010021202022000001021100220120001021101120002122012110021221211
00021112021200021202021221211001000102110022020211120102010112000212201120021221
21100021110201002212012001000211202201211000110212212110001210102200011100211121
020112212100200210120100221210100221101022000111002202200021110012221002222021
0122121002110020100212102221010000112000110212212110
```

Kosten: 372

Symbol	Haeufigkeit	Code	Laenge / Kosten
	24	00	2
e	18	01	2
i	16	02	2
t	10	10	2
n	9	11	2
s	8	120	3
h	8	121	3
c	7	122	3
l	6	200	3
r	4	201	3
a	4	202	3
u	3	210	3
d	3	2110	4
D	3	2111	4
w	2	2112	4
m	2	2120	4
g	2	2121	4
G	2	2122	4
o	1	2200	4
k	1	2201	4
f	1	2202	4
b	1	2210	4
Z	1	2211	4
W	1	2212	4
P	1	2220	4
E	1	2221	4
A	1	22220	5
?	1	22221	5

Tabelle 8: Codetabelle fuer schmuck00.txt

3. Beispiel: schmuck01.txt, $n = 45$

Codezeichen	0	1	2	3	4
Kosten	1	1	1	1	1

Tabelle 9: Codezeichen fuer schmuck01.txt

Symbol	Haeufigkeit	Code	Laenge / Kosten
	85	0	1
e	66	1	1
n	53	20	2
r	34	21	2
i	33	22	2
s	26	23	2
t	22	24	2
l	22	30	2
a	22	31	2
h	17	32	2
c	15	33	2
o	14	34	2
g	14	40	2
m	13	410	3
u	12	411	3
.	10	412	3
k	9	413	3
d	9	414	3
E	8	420	3
,	8	421	3
b	7	422	3
D	7	423	3
w	6	424	3
ü	5	430	3
v	4	431	3
p	4	432	3
f	4	433	3
S	4	434	3
I	4	440	3
ö	3	441	3
z	3	4420	4
V	3	4421	4
O	3	4422	4
F	3	4423	4
ä	2	4424	4
G	2	4430	4
B	2	4431	4
	1	4432	4
SS	1	4433	4
W	1	4434	4
R	1	4440	4
N	1	4441	4
M	1	4442	4
K	1	4443	4
H	1	4444	4

Tabelle 10: Codetabelle fuer schmuck01.txt

Codewort:

42322104431442441141010224100442243312003034414121204120423221044214414013003034
 3341312003141004430212230304120423221043434202012023333222141010431121410344141
 21204120440410043042221224012002223240123023242230304120420230232413341312004142
 2104344323121401300314112304233423120042322104420312124120044434414324333321200
 32121431342141204431411202402131204131200232233320413430202324302233321044403423
 1200440200442331233332220402340222130312041412001410432342141204202220042024424
 31234210424221044303034334131204133022204012042104231200442242212141313030201210
 42214241402442104442222102431411231204140420221210442041104222122204012042104421

34200442223241212324441211200401301402441204202220023430443312104234114332404313
42004444314313120203104421121424132240222002122204013020414121043443241121412044
03332043343032301031200410122201210434411233120203104202142431333212041410201411
10444131244112141204202303034322024023223332041031203332410313042104420411030221
42212042104434312304133441041024421020223332240222324034414121042431214120420222
00442321430323022204023401414223332244210401233332212214221200440410041344243024
123241200442314222141131214432

Kosten: 1150

4. Beispiel: schmuck1.txt, $n = 25$

Codezeichen	0	1	2
Kosten	1	1	2

Tabelle 11: Codezeichen fuer schmuck1.txt

Symbol	Häufigkeit	Code	Kosten
e	9	2	2
t	6	000	3
	5	001	3
w	3	010	3
r	3	02	3
n	3	12	3
i	3	002	4
s	2	0110	4
f	2	0111	4
b	2	012	4
I	2	1000	4
B	2	1001	4
"	2	1010	4
ü	1	1011	4
u	1	102	4
o	1	1100	4
m	1	1101	4
k	1	1110	4
h	1	1111	4
d	1	112	4
a	1	0112	5
W	1	1002	5
N	1	1012	5
F	1	1102	5
D	1	1112	5

Tabelle 12: Codetabelle fuer schmuck1.txt

Codewort:

```
10011002100010121102001011000021111000001011110110200110101112002200110011021211
22011001020020002120011000120111110002110101120000021110010200000001220102020122
1010
```

Kosten: 191

6. Beispiel: schmuck3.txt, $n = 9$

Codezeichen	0	1	2
Kosten	1	2	3

Tabelle 15: Codezeichen fuer schmuck3.txt

Symbol	Häufigkeit	Code	Kosten
c	34	00	2
b	34	1	2
a	34	2	3
	3	02	4
h	1	0100	5
g	1	011	5
f	1	0101	6
e	1	012	6
d	1	0102	7

Kosten: 279

7. Beispiel: schmuck4.txt, $n = 14$

Codezeichen	0	1
Kosten	1	5

Tabelle 17: Codezeichen fuer schmuck4.txt

Symbol	Häufigkeit	Code	Kosten
n	1	001	7
m	1	00000000	8
l	1	0001	8
k	1	0100	8
j	1	1000	8
i	1	00001	9
h	1	000001	10
g	1	11	10
f	1	0000001	11
e	1	011	11
d	1	101	11
c	1	00000001	12
b	1	0101	12
a	1	1001	12

Tabelle 18: Codetabelle fuer schmuck4.txt

Codierter Text:

10010101000000011010110000001110000010000110000100000100000000001

Kosten: 137

8. Beispiel: schmuck5.txt, $n = 40$

Codezeichen	0	1	2	3	4	5	6
Kosten	1	1	2	3	4	5	6

Tabelle 19: Codezeichen fuer schmuck5.txt

Symbol	Häufigkeit	Code	Kosten
	151	00	2
e	110	2	2
t	71	010	3
i	67	011	3
o	64	02	3
s	61	100	3
a	58	101	3
n	56	110	3
r	51	111	3
c	44	12	3
d	37	3	3
l	33	012	4
m	27	03	4
h	26	102	4
u	25	112	4
p	20	13	4
f	20	4	4
b	16	013	5
y	13	04	5
g	11	103	5
.	8	113	5
x	5	14	5
v	5	5	5
q	5	014	6
,	5	05	6
w	4	15	6
T	3	6	6
F	3	015	7
	1	06	7
z	1	1040	7
j	1	1041	7
S	1	105	7
H	1	1140	7
G	1	1141	7
A	1	115	7
;	1	16	7
9	1	016	8
5	1	1042	8
2	1	106	8
1	1	1142	8
-	1	116	8

Tabelle 20: Codetabelle fuer schmuck5.txt

Codierter Text:

10511203031011110400610201110000131011321110010301152100001010003201010202300402
11100120211010001011111212010011110103000301111001103112031161112311211031011101
20400131112401114001202321000040211100010102200103211021111010120030111001211120
10200110020111002012210010000121021011101102012001501101010202112010001202110100
01011110101111001010011300610220012021000101000002400120232000122010010211110000
11022300110020100001320020141121010120500101110300010102200100040301302012100002
11012023230010111120011002010001011001001120323000100200013200201411210101201204
00131110201310101301221130011500100020121120100110211000102101300131112501102112
10001204000132211000103011521100001304001140112440310111000011110001142016104210
60040211100010102200100132120111010120012101100200011110001510201112102001010120
12001202320001220100102111100001011112000240020141121010120012021000101130061022
00131112100211001000325201202130321100100001110000101012103201311110101112113000
15011111100010050010001011111212010112111200411211012010011021101000010111120032
40111102305000111100001021110310000024001510201112102001102122100100101111040010
11103001001124401112011211001000120211030110100110211010000402111000101022002140
11100010211012200024001311124011140012023210000031010400013200100010101010231130
00151110203000101022100200120211030110100110211010005000120111102101111000111102
01411210101201101001121000010111120032111011523001510201112102000310104000132001
1210023000100200121021011110112010211101110402001311124011140012023210011300114
10203021110406100000111100102103211100131110210311110103030111101030010101210302
11101101010203000111000001010221100011210023000100200120211010001011111212010000
21301001103112030012023210000100112013104121201000010020001010221002000111102014
11210101201101001121001600101110002012210310111001000120203013011110101010021110
11101012001011313111021011210200010020002013010101011110001010010001011101112010
01204001202031311201010101001102110101012002141321110112110122000111000013111210
02110010230001002003203021101000101111010102000101022001311110112010011121010130
11012011010040002400010102200032010102023113000150111101010120120405001000203200
10133011010011021101010120012023011110103001311102013012203100001011112003011100
12112100100230010111030010100131110201301220300024001201210110010001140111210101
0011021100001110000010111210101023113

Kosten: 3162

9. Beispiel: schmuck6.txt, $n = 34$

Codezeichen	0	1	2
Kosten	1	2	3

Tabelle 21: Codezeichen fuer schmuck6.txt

Symbol	Häufigkeit	Code	Kosten
	3	0001	5
	2	0010	5
	2	002	5
	2	0100	5
	2	011	5
	1	020	5
	1	101	5
	1	110	5
	1	12	5
	1	200	5
	1	21	5
	1	000000	6
	1	00001	6
	1	0002	6
	1	0011	6
	1	0101	6
	1	012	6
	1	021	6
	1	10000	6
	1	1001	6
	1	102	6
	1	111	6
	1	201	6
	1	22	6
	1	000001	7
	1	00002	7
	1	0012	7
	1	0102	7
	1	022	7
	1	10001	7
	1	1002	7
	1	112	7
	1	202	7
	1	10002	8

Tabelle 22: Codetabelle fuer schmuck6.txt

Codewort:

```
02200001110100100000020100012000120102220001101100001200110000010100000021000200
1002110201012111201111101001002002001200200100001202000210001020011
```

Kosten: 234

10. Beispiel: schmuck7.txt, $n = 82$

Codezeichen	0	1	2	3	4	5	6	7	8	9
Kosten	1	1	1	1	1	1	1	2	3	4

Tabelle 23: Codezeichen fuer schmuck7.txt

Symbol	Häufigkeit	Code	Kosten	Symbol	Häufigkeit	Code	Kosten
	13488	0	1	I	134	662	3
e	11480	1	1	L	126	663	3
n	7761	2	1	R	115	664	3
i	5018	3	1	G	106	67	3
r	4011	40	2	T	102	8	3
s	3507	41	2	Z	79	48	4
a	3307	42	2	j	72	58	4
d	3272	43	2	J	68	647	4
h	3213	44	2	-	66	657	4
t	3174	45	2	;	37	6650	4
u	2609	46	2	)	32	6651	4
l	2448	50	2	(	32	6652	4
c	1991	51	2	—	28	6653	4
g	1665	52	2	:	28	6654	4
m	1521	53	2	O	27	6655	4
o	1304	54	2	U	25	6656	4
b	1141	55	2	]	24	6660	4
,	1077	56	2	[	24	6661	4
f	1000	60	2		21	6662	4
w	941	61	2		21	6663	4
.	831	62	2	P	20	6664	4
k	798	63	2	1	16	6665	4
z	631	7	2	4	13	667	4
S	498	47	3	3	9	68	4
ä	455	57	3	?	8	9	4
ü	434	640	3	5	7	49	5
ö	399	641	3	2	7	59	5
v	361	642	3	!	7	648	5
p	300	643	3	6	6	658	5
F	300	644	3	0	5	6657	5
SS	278	645	3	Ä	3	66660	5
D	277	646	3	q	3	66661	5
E	243	650	3	8	3	66662	5
V	242	651	3		2	66663	5
W	231	652	3	Q	2	66664	5
N	218	653	3	9	2	66665	5
K	206	654	3	7	2	66666	5
B	193	655	3	Ö	1	6667	5
M	159	656	3	y	1	668	5
A	159	660	3	x	1	69	5
H	152	661	3	C	1	649	6

Codierter Text: (Zu lang, siehe Ausgabedatein)

Kosten: 134559

11. Beispiel: schmuck8.txt, $n = 321$

Codezeichen	0	1	2	3	4
Kosten	1	1	2	2	3

Tabelle 24: Codezeichen fuer schmuck8.txt

Fuer die Codetabelle, siehe in den Loesungsdateien.

Codierter Text:

```

110002102201013111030313000000210122110000023101021232040111102020101133031000121
21110030102301311110001113033431123030323210010332100302221100211004001440010000
10210010100000022000301113011010100341110110000131143443410100210302100144001000
01022013100123011130130201300311403201111321000001211003002111000000211130212103
11011400012000311111023211341221011012101230110131101030000002004101003102000021
11231110111022101412200002010012021210231134010140210014001002201214410120310000
31033133132401112101023010004032030100011310411114111201233013003000112011011010
30000101101000213000110011014110311324002030100400111201233013003203001010000101
12001203100113300002001001111012110231231310240144002104120101140010300000001303
34113300021010000210232200010230003111140004000010111103004103141100130002230011
10111101110110112202030031101102001033004010112001100012330130030001100100131012
03100111010100021000020201100300014011323211130143100002030111131123111022112012
11012103433013003130212131101021101130001031000211011102120010111101030314010011
12120000040001101010110010113120010032111130302000040104212210301020111111110340
02011012124121001021124200110111302104200011033110310100000200310011010000011144
01010230021010130040321110032001110101301011000012010100010040200131040003044304
13131001121031013003300301100031132431020210210001003103011002000034002003101110
01010310211300010322133020003010111101100101033131111120144002003110001241110101
02101432111302000330011101412011111023001203400203000110210003012330130030113001
11300111012000011201101011001202010000100310010011104314212113100203010111211112
00110031140012110401201233013003211401011011121000011010121011210320200010101103
00120000313140111032010400200101300320100021000130101000201114001040312213400111
12014001111214100130011330201300301004001010113111101312040010121401030200001113
11041013021030221320012131040010110330410130310111001301210004023211001011000300
00120001130112001213100110401111211210101010212311313100221000104002010010213001
14000210204010030143211211120020010013204113001120411302203111121112000010040300
01101110220110121101120310111002010301000310010212021113102011201421040112011323
14040201132314113010302004141311041013103102110011000400021101310104101142002111
11112112021421020222000130010203000110111022011012011203010110200120121404022200
01300102030010120121211030010020120130410011132000310000310110310100113123301300
30001110013000011121400110313041002021002011100211000131011313400011140020013031
11002220311040334

```

Kosten: 3287

12. Beispiel: schmuck9.txt, $n = 674$

Codezeichen	0	1	2	3
Kosten	1	2	3	4

Tabelle 25: Codezeichen fuer schmuck8.txt

Fuer die Codetabelle, siehe in den Loesungsdateien.

Codierter Text:

```

20010300001000120000000300011010001010001010013010001100100000030100010002030001
10100010100001301010001022002230103001002010001000001310130110000110100100001000
30000122101010001001002210001022002230011000101022001100210033011110011000000110
20101011000100100113201202000002100010000011011012200030010010202003000111100310
00020002320000010000000301000112100131111020110223300301010022310011000002100010
00110221301101001000101001003000000000020000000020001103100101330210010010030000
00001103100000110020101030220110202030010011001001300000020011100022101010201220
02232000011030200000001100202002010010000111310020233121000300100101000000001103
10000011002010103022101110002120000010110000000220013111110000100110002003001000
13000100000022110010002220002301011300200010001020002003301110210011201100000210
20010310200300010210031001010031001002000112000010000020101001000003000110031320
00100300102102310031100001000112210101000001001000000000012102131030110000002102
01110020020100010001031000001101011101321110000100201000000030010220003111002000
01001101232012311202120203110002011000010210031001010030000011010021010100103101
02000000130220110202002111213030000300122100131102000320111020003200001030000000
2010000023210010001210000311122010020221000001300200000000131000300302001001101
20002301001002100101021000001013101000001000201100002220022100100210010100011000
20030022100021000200301131033010000200012030100011020000000013010101222311202111
00100100101112110201011100001131000131100102102010010100300030021012302010011002
00000030031100311100110110000002003001000000200200000100003100030100101003111201
00111301221102000222031101020100000000210200010011000300211112200220000010020021
01230001110000101113011000000210100110002000000012010010212100032110001001031010
12000000120020010003103011020203000010001110120000132100010210031001010030000011
01002100200000021210000101210111010211101001101001102002010001033112021201001200
02003000102200223010310132002200011000021001100010102200110021111010010003000110
10310002210000002000110110213103001310202201101000120000010111100100100101101012
01201130000001110020000010121000220102111000010000001101200200000110011102011000
00021000000001303000101120100120110101320022320303100001003000132000200013000000
11121002200000002000201001230000000110010110201202003000021230020100013000020000
10102111000030012210123112021000222000200000210101100100011000010310001003212020
30000100000011012002011000100201000010100200021201002020101301010123002111000020
1000000300000000000000001030113000100012001100100100022100000200000101001110300231
03101012000000120020020001002202000001113011011212001110321010100010210031001010
10202001101100000100001100003320000231111000003001220100110020110000002002101102
00000100000003010001121011301000021020013311200130011012002301100100001300001010
10320010100113201100013000301000100111110000100010100000101002001001000011102001
00001023100301000010200211202120210120120011103011002011031000000020300000020122
00030001022002230310001110020000010000210003120000122001301010110101103002210220
10310101010010201210011000000110210200100010021000030000021000100000110110103000
0110121000001102001000000100021203130101012302200111000010111312021100022100000
20001101010101001100001100001000213011010101200000012010120000010000000303000100
13001120203000101120100120300000210011030001001200001100011011010011000200301101
01122133321000020002320001031020001010132012223112031300131110210112011100100001
0100013013000021010320002101000032300000021200303200010001101101010120103000003
10000001010030003002112110000103110001110000001020200300011110030300112011000002
00200200002310033000301100010001101000110011030010130100020010110000100000001012
01100000011110103000110203001001020000000110100011011002102100011100111000013100
00312000102200223000000100002003101002001100202000000001101000010102111201001113
11010011200210033012021000222000200103102003201031100000130100012000002001000000
00311001002200031000100000101110010200211210100031001110110000010011000000100113

```

00010101320010000300001001000020000000001001000300330122000201301030010110000201
10010000021013010300010210030131020001022002231000120000020010100100002110102200
00120001010010020001001000100010032100000010110000000111000200301100010130100020
03001000100000203031202001202000000200001100020102111110000031101030013022001100
00002120030010002000001000100200200011000100110002011000001100001011011020000112
11000030010200211112010011210123112021120001200022020311012001000100010100003010
10000010200010001020300000111101030001102001100100001200101010031000020001110120
02322010020222001111303000201230020010000002020100000120120022030001231120210112
0111001003110303110012101000010001000000000001110210131002201300100301000002013
01031002123100201010310131002201300100301000002013101100100100022100010003110001
20010200211220000100100001010000110000000212003030000012000100000010002001020101
11200012000101031130030200100032001001011000000202110021021100101000010110102111
00110001022002230310001110020101000100033332000012102001000100100001010100100001
10001003030000300122100000000100112310031031202100022200020010020203203000021000
13000111102001033100310312000110001110002010101000021020001000101000232101010002
20110000003000001200001201010010011000012002201222311202101111010110000010022310
02012322000300010220022300000010000200310100211213001010001000000200011002001000
00100020000000012310030100023100203002000000100020000001120002011002000101000000
00011010022200030001022002231121301010010011000000200011002001000001000200000000
12310030010002000001000103000000200200011000000020110100100110012001100021202100
01210220210120120000103000020000001011100302202000001000200000000000100011000022
10000001030100231120210000200110113030001012000000102301210100210011110201001030
00000102100000010021101030003002111100003001221000102200223101002211000010200000
00100200102002100330111000120201013011011013210111103110010100001011002010112100
20201021110022030101112311232012223112031300131110210112011100111101010100000100
20022320000110200301301031002101301000000000101000000200113000121001020100010300
01011020100110020010000310000310110010000120001011020100301000000100100000310031
02011000000202110020311001010000101121100130100210102003020001010013100011011002
0020000001110110000002003301112010011002000000100202200001010310120001202101003
11030311021000011001200230111202200011211010113101010010000020002003320021111001
13000122200030110012020010320100110000010003000000003100130011201030200010022000
00120020110120000103010211120100112100021202101111010100102001020010322012020130
00001200200000132100000111100003201000101100111010010101001002000100000001000201
01011110010011000012000123112220003010001003200000020101123120001000202200133112
10100101010010000020300200000000210012200010012101113002000000101231201000103011
00001100210020001000000300113000120022030110000001001011012200300200200002111100
01101100213012020100021200111011020100211301230102121021200113000120000230010000
00121000000010211300330120210112011100010020203203000021000130001110101000102200
22301130100311030300000311003022000101000000200102001010100300000010120010000022
10000000001021011120021003301200220303011000011300000120101112000120000031220100
11102012223112021020001010013100011011002102020101212112000230010011100200100101
00300311001001001100030000132101001000102010201000103011000100101110013300001010
1100110003110000200101001001000020000000002002000000002130333201020000300122100
00000021113010010133010031001000032000002100300010201130100311030020010120010210
00300300000312001110301100111101010000113000010100203000001103002010001131131112
01001110213103012223112021000001000013332010000210201000200222200031001031011000
02002002010002330100100000130011010101001000002000212000000100010301000001002002
00011123100201020000010002210200010100131000110110020301000200000010200200320001
21030000031112000013000012220301000001101301000001123010000101010300002101000000
00000200320001202010200110110110010010011000101112111130001310001000320000100201
03110000000220000110001110010103100220310110000000000101002011010033100330111220
30000300122110100300020022000012100100110201222311202101120111000100000110001000
03001110101001110100002011000011020000100020302011210000020302010100200102202000
12030000300122100010101102111000210100011030110000110100000000022200030100101110
20100010021110100110200100000100100101102020021000310200130003010001001111002203
21101122100000311301230011002010002100230022202002100111002000221110010103200000
00301310030020200001000101020110100000300010000032100001033000030021111000000110
12000100203000030012210000000110100002200200100021001030201000102101021202120200
21002001000200002003000110002221000030201110211101130100311030101011000100202032
03000021000130001111021002003010000000303000013131001230100000001130120023101201
20000002030000302200221000100012221000001000000000011300001001213200100200011000

00000123112021100011011002010101020100102210000002103100000120311010121010000010
21123112021202201220003000100122000001002010001000010201000311000110010100101000
00001013000022112130100000011100010010000112103031010101200310233300033012021011
20111011001030200210210020110021002010000010000201012021011201110023201201102010
03110313010202020011011110101000000301002200100100221020120320100110020000000331
12021023201201131010100022111101213000212000301100201300000120101010000001001000
00310032102001000000010302010000200302001103112010202020112011100010020203203000
02100013000111010120010000100000011020020000001100032221010001111001101100010001
10001001000032000000000002001001010030110011120220001121000310202010003010000203
00001321010011210000310111000212021101010120032100001122200211200000200021000100
10332010002100010321000001100030010011012112022102131030033000301101010013301330
12011110101010000000101300002221000010113211213210100001001210100110002002000003
10001011002001000322000200300021123112021101001101130121001000020001122132010010
20221001000310200130220110011110101202201212222111020000303200100022000010030011
20011022200100030020000002321100001002000200103310030100311030300001321011123112
02101120111011200012001001010000101000130101012223112102011010000120101100000000
00211000001012000300102002101101101100000120300001301010102132120001020000100332
01231003301202101003001001002000000002020010031302000300200002003301201001001001
12212112022030000110021211021020111202200011203010011121000023022100100000002201
10110110100031001022100010112231003000000010110121310312021020000001200001011001
10101010000130001012002000000102200121201003000000101002002130110203000021200011
00100300010020001110020010010022100000000000210301100000020130110110100031001020
31000020001221100103000103000100011111001110000001210000010010020000212021011201
11002320002013010300101100002010130010110103101310022013001003010000020130311000
30000102030000110113010201202130001030110110003000010203020010010011000012001100
00003300000010100010200101011031300021000011001210001010212000201301030010110000
20101300101101010001001012000000020120200120000001000011100202010120102032101101
03310011001301212001001000000100011210022110003000010200200200101000010330130002
10000110012220003020201120111000100202032030000210001300011101010000200030001000
20020100002101020000300122100000000001103011001003000000101002002010102000300023
32000101020101202102310001200000000003000002200101020002112010200110110010021200
02010100010130000111001000210011120000020130100001223001100001103000211201020210
10021200020320100110101103310011200020130103001011000020101300101122000302001000
00001001103130002100001100121020110201030100311030302000202000010100121000200001
10002003100110100100000120302001020000003103100133000203012000123000302001022112
01020210013111001110301200232000201301030010110000201013001011010101002120002031
00102210200100100000001112022210120221120002003012021000002013000001101102001303
00003001221000010011000000100101211011010003100102030022001113110012001010002131
03111001130001203010111220100112111000000102221100120012001010110320100111301020
0003021001300111300100100100001001303110012101000010001000000000011121333000320
00201301030010110000201013001011102020011000010111321012310033011120100110020011
31122221011020010132000010200001000001210200011110001000021120102102011020203110
10121000100002010011301020010101103000020200011031001011210002003010021200020310
10010101001011010000000121300131110000110120013002000000220010102000211200021000
0110012020202030001121011002210101011000232012200030000030030022100110100101000
00012032101102021012012000010010100001020020110100000001210002033001021110301010
11010001210020000020101000000010003010220012130101012302200113000120000101300001
10031023032000322201100030003002100330120210112011100232011102000301010100003203
00010020103010000100010000010301301000101013100302102012100301000122311001102002
13011001002120002031002310000210300220011011000000200210000013030100111000010001
00020000100100111001100020033011201020212011102000301011002010010030001002010302
03100010000210131002201302100011010010000100030010100030000020001012020010010012
02002001021012130102021202000200320111020003030100003203000100201030010110000201
00100201000011101301011011001000001100200200120010231003301111012130102102011000
00102031000012301200111032100002020001103000100031001110011000200301003110300000
1001100002003030200100001020200000023000200002003301200012300033011201020202011
20111000100120200200110201210030100012231100110200201010000022001010200200001000
01202010003011000000000001320000300120120110000000001200200102321000311100003001
201220000000110000002002000021202120200210101020002020000101001210002000300120100
01020100021002100300010000201131010001311210011030000200221000100010001021110100
00100220001300200013000301002000201101000000012102001001121002201111000310201000

0320300010020103001002020320300002100013000111001111010000000001000210121112010
01110200133112001100020030022011020200210101100021000200300010130000111001000210
01112000002013010000122303000002200101020302001220010121301010123000012100000200
12001130001220100110020000010032110000210002003301202120200210011010110001122101
00003203000100201030010110000201001002010000111013010101101000121002000102020002
02000010100121000200031101000000012210000200111301112002100030021122010011031010
11000103002000101000021002000211100000000101001200021200030002320110101000002000
23200100111000000002000201210200031000000130000000232102300100010122001300212001
10100212000221020013100022000033100332000120010230020200110000003300110101010220
00100002100002311110000030012010200101010020010021031121130033000301102010021220
00300100132110111000030012001100020033011100000201301101101000310010221303313002
21010000001000200010301232110100110100002030000102002000011001000200100012001010
11110000103000100020300131112010011002011000100003222100021202101120111020100120
00212020002000020001121102031101321002220203100110021010100002020010002000011100
0101003100111001101100100020022200030100311032110100010003022001101112310031031
20000200110113030100020101100000001200002000110230120210111100030101010000201010
00203001012111000201100001100011100020301001001001100111030020100100010100100021
30000100010000232000001022001011010220200120001200112001010010010300000011300012
31120210112011100232011102000313020010010002300102111201001100200200100001001000
00103200102200003012302200111001231120313010200003020200110200020200001010012100
02001020000003311202102300102312023220103110110000012030100100100111030100210112
21333210002011100013002020010110023020001300020030001100100020201100101211120100
11102131031120102220003012021200020130103001011000020101300101101031013100220130
0100301000002013010101102010311010102001101113210123112100010002011001002120002
03020001111000100020110001010110030001200110000003310032120121201000201210200032
00000001100030033200012001023011200021200003020201120111001020100102211000202002
11112003010101200102202001200012311203130110001002020320300002100013000111010111
01002101010300020110101003000012000020010300100110001010010101000001030021120010
00110001001000001030100000012030010000000030001100133011231003000100000210002010
31000000001000010010210010010020100001001000210111000200330111000001310100020020
10010310310111000212021000020200100020310000121001001010101001001101200103022001
10010000131232011000000103002120201000213112000011200000120300100000000301020010
02020320300002100013000111030000011200010002002000011020000100020301000020001001
02022001100002030300131102201110221131031201001101211202221000101102010001021010
21202101120111002300000131010002102301102200111012110000013003130102020200112010
31000000010200001020120010100320000010000120101000000331120210230010020100010000
10030001002013010310030103010000002200011001003210000331003000101000200001010010
01010001000110021000000000200010122111011003000300211120002120202011201110023010
02002101002002103020011101000000000010000101201032000000100001110020020000110102
11121010100000201010220012003000000100201001000002000020111320010220000301230220
01101100023001200222000020010000213012000301230030001210210001010210013012120101
00230012100120012020201111010100000000011002000010113000301022001100103320010012
02002210000000202100010112021011201110023001300130110001301212102000010211003022
01113110030220001001011010031002300000001120120010200212001101130010100200000100
12000120011100123011201020000302020010012020020101202001200000100311301210200030
00010003001000010100203302202011001012003000003101020030201210030100012231100110
20020020210010302200110010210102120210112011100230010200003001201102012010101010
200101010031003010301000000220001100100310211011003120313010202020002020000101
00121000200010100100202032030000210001300011103100002000122100010111030000010100
02201210200033320001010201012021023000000001020100022000120011100123011210102101
22201100100010110030200323000010000100010030300000023000010020010201010210000013
10100020310021013110000003013100201010100002000021011020102111210100000130002112
01022000120212020021210010130001100101000110320103000010021001120003000230100210
13000022002000000300332000120010230021002211021000002101201000101100302003230000
10000100010031301021020110012101200111001012200130011000200330111201001122021202
00211330003000000002203000030012210200000200101002130110110002110102210100000000
10010100001001020021120010003032001000101130121010110100130020000231220220000110
00011300330111021000010021001120003000230021000113210000030000111000120202011201
11000100202032030000210001300011101010201101001210200100032120000200100002002010
00023300033012201001100201001230101110002003200223000100223002000000302011001012
1202101021110011110001100101001210101001001000011010101101000000032000003210001

00001100002000200201100000210003201101200012100113102000001010103000030012213330
00302001022011011000020010100121010120022300010022303000000201010000000203001010
00002200030000001330200122000320012200030300011000300211011210200000110010000010
00020020001000000020001000130002122203000030012210200002310033011103102000003113
12021011110101023100210130000220101100021000200310001020003012300202001130001211
00000130110110100030310101300201001230101110002120210010010133030113103012000000
11000002300001001030010001032000000300200000030201100101200330111001101102020021
00031022010012000000331003301110000300122110012010020021120210112011100231002101
30000220101100021000200310001020003012300211000210002003021011110003010101002002
10100200210302011210000000003000000020030103002101230100002000230310101102013002
01001003101021202101002002101002002101012020310101101001300201002000100200101021
11021100210130000220101100021000200310001020003012300202011201110023000010021001
12000300023002100011321000003000011100012010001302020200020200001010012100020001
0133101000100103103320001010201012021023011300000000101012201001100200010311110
00100200001313010201201100100010110030200323031101101000302002121102010102020112
01110023113113011020200121301021000101020200110010020203203000021000130001110101
11000000012001000022000311100200001001101222311202102300022200022010300100031021
00210130000220302011210000002200010302011001012000300211110011010110012310033001
22231121300021000011001202020112011100230122101101002101300002210220000200100002
00000010030001231003201001122100010102020201210030100012231100110200210200011200
20010100102101200012021023000113010101002120002002220210120121001001102000200330
12101200012000030110001000101000020020022020201120111002300102001301102020021010
10023001213000210000110012220210100200210100200211021000131030033011100133000012
01010000110011102201121131110011100123011201020000302020010020203203000021000130
00111010100102022001012012320110001000020000200000001021010211100003001221000000
3311202100100032110000001100020030101300003010001020000020000000000000000020010
01010031110120011100020121000001120001000200102002100330122010011002010012301011
10002120300012021000222000200000000010030201100200000031110011011000000101022100
00101021000002001200110002122000000002010000110100001000100210000010000002000303
10100110000121211021301010123002010002100230110100001000201301000011020122300010
00010210201220001120002000100001000200000002200010310101110001011200003001201210
01001102000200301200032001001011000101300011001010001103101001101130121002101300
00222100010001000102111200002001000030000113010110000010230000030012002201222311
2021

Kosten: 36597

7.4. Fazit der Evaluierung

Anhand der ERgebnisse der Beispieleingaben sieht man, dass das optimale Verfahren eine akzeptable Laufzeit erreicht. Weiter sieht man, dass die Approximationsverfahren mittelmäßige Ergebnisse erreichen, wobei das erste Verfahren in fast allen Fällen die besten Approximation erreicht, das dritte Verfahren die zweite beste und das zweite Verfahren mit Abstand die schlechteste. Dafür sind alle drei Verfahren verglichen mit dem Verfahren für optimale Lösung extrem schnell.

8. Quellcode

Nun folgt der Quellcode der wichtigsten Teile der Loesung in C++. Dabei wurde *cord* als pair an zwei long long's (wiederum definiert als *ll*) und *ctrip* als 3-tuple an *cord* definiert.

8.1. Huffmans Verfahren

Als erstes wollen wir die Implementierung der Loesung fuer den ersten Teil der Aufgabe mit einheitlichen Codezeichenlostern, also Huffmans Verfahren, betrachten.

Die Darstellung des Baums geschieht durch die ein struct *Node*:

```
// Node.h
2 // Node in the huffman tree
struct Node {
4 // The character associated with a leaf; 0 if its not a leaf
  char32_t data;

6

  // This will be the sum of frequencies of subtree
8  int frequency;

10 // Pointers to child nodes
  std::vector<std::shared_ptr<Node>> children;

12

  Node(const char32_t value, const int frequency);

14

  // Function adding a child to this node
16  void add_child(const std::shared_ptr<Node> child);

18

  // Comparison for Nodes
  bool operator<(const Node& other) const;
20
};
22

// Node.cpp
24 // Implementation of Node.h

26 Node::Node(const char32_t value, const int frequency) : data(value), frequency(frequency), children() {}

28 // Function adding a child to this node
void Node::add_child(const std::shared_ptr<Node> child) {
30     children.push_back(child);
    frequency += child->frequency;
32 }

34 // Comparison for Nodes
bool Node::operator<(const Node& other) const {
36     return frequency > other.frequency;
}
}
```

Der eigentliche Algorithmus sieht wie folgt aus:

```
// Create the huffman tree
2 Node root = create_tree(letters, r);

4 // Create a code book from the huffman tree
std::map<char32_t, std::string> code = create_code(root);

6

// Compute the encoded text using the computed code
8 std::string encoded_text = "";

10 for (char32_t c : input) {
    encoded_text += code[c];
12 }
```

Hier ist *letters* ein vector der Paare aus Integern und *char32_t* (also Zeichen) enthaelt, wobei jedes Zeichen aus dem vorgegebenen Text einmal vorkommt und der Integer der jeweiligen Haeufigkeit entspricht. Die benutzen Funktionen *create_tree* und *create_code* sind wie folgt implementiert:

```

// Define code letters
2 const std::vector<char> code_letters = {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z'};

4 // Utility function converting a vector of integers in the range [0, 35] into
  // a string using the code letters.
6 std::string to_string(const std::vector<int> &vector) {
    std::string string;
    for (int x : vector) {
        string += code_letters[x];
    }
    return string;
12 }

14 // DFS for traversing the tree and creating the code.
  // Here code_word_vector is the code word created by adding the code letter (index)
  // of the childs on the path from the root to the current node.
16 void dfs(const std::shared_ptr<Node> node, std::vector<int> &code_word_vector,
18          std::map<char32_t, std::string> &code) {
    // Amount of childs
20    int childs = (node->children).size();

22    // Check if the current node is a leaf (ei, has zero childs)
    // and make sure its not a dummy / fake letter having frequency 0
24    if (childs == 0 && node->frequency != 0) {
        // Set the resulting code word for the current character
        char32_t character = node->data;
        std::string code_word = to_string(code_word_vector);
        code[character] = code_word;
        return;
30    }

32    // Otherwise recursively visit all childs of the current node
    for (int i = 0; i < childs; i++) {
34        // Change the code word vector according to the index of the current child node
        code_word_vector.push_back(i);

36        std::shared_ptr<Node> child = (node->children)[i];
        dfs(child, code_word_vector, code);
38        code_word_vector.pop_back();
40    }
42 }

44 // Function for creating the actual code (ei. a map from characters to their codeword)
  // given the root of the huffman tree.
46 std::map<char32_t, std::string> create_code(const Node &root) {
    std::map<char32_t, std::string> code;

48    std::vector<int> code_word_vector; // Initially empty code word
    dfs(std::make_shared<Node>(root), code_word_vector, code);

50    return code;
52 }

54 // Function for creating the huffman tree, given a vector of letters and the amount
  // of code letters r.
56 Node create_tree(const std::vector<std::pair<int, char32_t>> &letters, int r) {
    // Priority queue sorting the Nodes by their frequency, such that the Node
    // with lowest frequency will be at top
60    std::priority_queue<Node> queue;

62    // Initially add all letters to the priority queue
    for (std::pair<int, char32_t> letter : letters) {
64        int frequency = letter.first;
        char32_t character = letter.second;
        Node node(character, frequency);
        queue.push(node);
66    }

68    // Add dummy letters (frequency 0) in order to achieve queue.size() - 1 = 0 (mod r-1)
    while ((r-1 > 1) && (queue.size() % (r-1) != 1)) {
70        Node node(0, 0);
        queue.push(node);
72    }

74    while (queue.size() > 1) {
76

```

```

Node node(0, 0);

78
// Add the r nodes with lowest frequency currently in the queue to the current node
80 for (int i = 0; i < r; i++) {
    std::shared_ptr<Node> child = std::make_shared<Node>(queue.top());
82     queue.pop();
    node.add_child(child);
84 }

86 queue.push(node);
}

88
// The last node in the queue is the root of the whole tree
90 Node root = queue.top();
queue.pop();
92 return root;
}

```

8.2. Gemeinsamer Quellcode

Da alle Algorithmen fuer den zweiten Teil der Aufgabe (mit unterschiedlichen Codezeichenkosten) denselben Aufbau haben - zunaechst Einlesen der Codezeichen, Einlesen des Textes und Bestimmung der Haeufigkeit, gefolgt von einer jeweils unterschiedlichen Art und Weise, n Codewoerter zu erzeugen, und schliesslich der Zuordnung von Codewoertern zu Symbolen, abhaenig von ihrer Haeufigkeit - wollen wir hier die gemeinsamen Codeabschnitte auffuehren. Diese sind in der Implementierung in eine eigene Datei ausgelagert.

```

1 // Define code letters
const std::vector<char> code_letters = {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z', ' '};
3
// This map will point from a character to its index in the code letters vector
5 std::map<char, int> code_letter_map;

7 // Create a map mapping the code letters to their index in the code letters vector
void setup_code_letter_map() {
9     for (int i = 0; i < (int) code_letters.size(); i++) {
        char character = code_letters[i];
11         code_letter_map[character] = i;
    }
13 }

15 // Utility function converting a vector of integers in the range [0, 35] into
// a string using the vector code_letters.
17 std::string to_string(const std::vector<int> &vector) {
    std::string string;
19
    for (int x : vector) {
21         string += code_letters[x];
    }
23
    return string;
25 }

27 // Function computing the costs for some code word, where the code word is given
// as a string and we have costs for each index of the code words
29 int get_costs(std::string& code, std::vector<int>& costs) {
    int cost = 0;
31
    for (char character : code) {
33         int index = code_letter_map[character];
        cost += costs[index];
35     }

37     return cost;
}

39
// Function converting a map of letter frequencies to a vector containing
// pairs (amount, letter) for each letter in the given map
41 std::vector<std::pair<int, char32_t>> get_letters(const std::map<char32_t, int>& frequencies) {
43     std::vector<std::pair<int, char32_t>> letters;

45     for (auto p : frequencies) {
        char32_t ch = p.first;

```

```

47         int frequency = p.second;
         letters.push_back({frequency, ch});
49     }

51     return letters;
52 }

53 // Given a vector of code words, return a vector of pairs consisting of code word costs and code words
54 std::vector<std::pair<int, std::string>> get_code_words_and_costs(std::vector<std::string>& code_words,
55     std::vector<int>& costs) {
56     // Find costs for each code word
57     std::vector<std::pair<int, std::string>> costs_vector;
58
59     for (int i = 0; i < code_words.size(); i++) {
60         std::string code_word = code_words[i];
61         int cost = get_costs(code_word, costs);
62         costs_vector.push_back({cost, code_word});
63     }
64
65     return costs_vector;
66 }

67 // Create the actual code (ei., map from chars to strings) given the letters and code words
68 std::map<char32_t, std::string> create_code(std::vector<std::pair<int, char32_t>>& letters,
69     std::vector<std::pair<int, std::string>>& code_words) {
70     // Sort the letters by their frequencies (biggest first)
71     std::sort(letters.begin(), letters.end());
72     std::reverse(letters.begin(), letters.end());
73
74     // Sort the code words by their costs (smallest first)
75     std::sort(code_words.begin(), code_words.end());
76
77     // => Now, the letter at index i will have code word at index i
78     std::map<char32_t, std::string> code;
79
80     for (int i = 0; i < letters.size(); i++) {
81         char32_t character = letters[i].second;
82         std::string code_word = code_words[i].second;
83         code[character] = code_word;
84     }
85
86     return code;
87 }

```

8.3. Approximationsverfahren I

Die Implementierung dieses Approximationsverfahren ist logischerweise sehr aehnlich zu der von Huffmans Verfahren. Hier wird fast dasselbe struct *Node* verwendet, mit dem Unterschied, dass die chars an den Blaettern nicht mehr gebraucht werden, und deshalb nicht in *Node* vorkommen. Der Hauptteil des Algorithmus, der die Codewoerter generiert sieht wie folgt aus:

```

1 int r = costs.size();

3 // Create the tree
  Node root = create_tree(letters, r);

5 // Create a list of code words from the huffman tree
7 std::vector<std::string> code_words = create_code_words(root);

```

Mit der Funktion *create_code* kann man daraufhin den eigentlichen Code generieren. Die benutzten Funktionen sind wie folgt implementiert:

```

1 // DFS for traversing the tree and creating the code words.
  // Here code_word_vector is the code word created by adding the code letter (index)
3 // of the childs on the path from the root to the current node.
  void dfs(const std::shared_ptr<Node> node, std::vector<int>& code_word_vector,
5     std::vector<std::string>& code_words) {
6     // Amount of childs
7     int childs = node->children.size();

9     // Check if the current node is a leaf (ei, has zero childs)

```

```

// and make sure its not a dummy / fake letter having frequency 0
11 if (childs == 0 && node->frequency != 0) {
    // Add the resulting code word to the list of codewords
13     std::string code_word = to_string(code_word_vector);
    code_words.push_back(code_word);
15     return;
}

// Otherwise recursively visit all childs of the current node
19 for (int i = 0; i < childs; i++) {
    // Change the code word vector according to the index of the current child node
21     code_word_vector.push_back(i);

    std::shared_ptr<Node> child = (node->children)[i];
    dfs(child, code_word_vector, code_words);
25
    code_word_vector.pop_back();
27 }
}

// Function for creating a list of code words given the root of the huffman tree.
31 std::vector<std::string> create_code_words(const Node& root) {
    std::vector<std::string> code_words;

    std::vector<int> code_word_vector; // Initially empty code word
35     dfs(std::make_shared<Node>(root), code_word_vector, code_words);

    return code_words;
}

// Create the huffman tree
41 Node create_tree(const std::vector<std::pair<int, char32_t>>& letters, int r) {
    // Priority queue sorting the Nodes by their frequency, such that the Node
43     // with lowest frequency will be at top
    std::priority_queue<Node> queue;

    // Initially add all letters to the priority queue
47     for (auto p : letters) {
        int frequency = p.first;
        Node node(frequency);
        queue.push(node);
51     }

    // Add dummy letters (frequency 0) in order to have queue.size() - 1 = 0 (mod r-1)
53     while ((r-1 > 1) && (queue.size() % (r-1) != 1)) {
        Node node(0);
        queue.push(node);
57     }

    while (queue.size() > 1) {
        Node node(0);

        // Add the r nodes with lowest frequency currently in the queue to the current node
63         for (int i = 0; i < r; i++) {
            std::shared_ptr<Node> child = std::make_shared<Node>(queue.top());
            queue.pop();
            node.add_child(child);
67         }

        queue.push(node);
69     }

    // The last node in the queue is the root of the whole tree
73     Node root = queue.top();
    queue.pop();
    return root;
75 }
}

```

8.4. Approximationsverfahren II

Das zweite Approximationsverfahren benutzt das folgende struct als Knoten in dem Baum, aus welchem am Ende die n initialen Codewörter eingelesen werden sollen:

```

// Node.h
2 // Node used in tree generating lowest cost code words
struct Node {
4     // The code letter
    int letter;

6
    // The cost of the code letter
8     int cost;

10
    // Pointers to child nodes
    std::vector<std::shared_ptr<Node>> children;

12
    Node(int letter, int cost);

14
    // Add a child to the childrens
16    void add_child(const std::shared_ptr<Node> child);

18
    // Comparison for Nodes
    bool operator<(const Node& other) const;

20
};

22
// Node.cpp
24 // Implementation of Node.h
Node::Node(int letter, int cost) : letter(letter), cost(cost), children() {}

26
// Add a child to the childrens
28 void Node::add_child(const std::shared_ptr<Node> child) {
    children.push_back(child);
30 }

32 // Comparison for Nodes
bool Node::operator<(const Node& other) const {
34     return cost > other.cost;
}

```

Der Hauptteil des Verfahrens sieht wie folgt aus:

```

1 // Create the n least cost code words
Node root = create_tree(costs, r, n);
3 std::vector<std::string> words = create_words(root);

5 // Create an prefix free code using the code words
std::vector<std::string> code_words = create_code_words(words);

```

Die Funktionen die dort benutzt werden sind die Folgenden:

```

// Create a tree which will later generate the n least cost code words
2 Node create_tree(const std::vector<int>& costs, int r, int n) {
    // For each Node in the queue we know that it has some potential child such that the results costs
4    // are that of the integer in the pair.
    // That child will always be at index node.children.size.

6
    Node root(-1, 0);
8    auto pointer = std::make_shared<Node>(root);

10
    std::priority_queue<std::pair<int, std::shared_ptr<Node>>,
        std::vector<std::pair<int, std::shared_ptr<Node>>>,
12        std::greater<std::pair<int, std::shared_ptr<Node>>>> queue;

14
    // Add the root with the cost of its first child to the queue
    queue.push({costs[0], pointer});

16
    // Counter will store the amount of nodes in the tree,
    // ei. the amount of code words created by the tree
18    int counter = 1;

20
    while (counter < n) {
22        // Get the current node
        int cost = queue.top().first;
24        std::shared_ptr<Node> node = queue.top().second;
        queue.pop();

26
        // Get the index and cost of the next child

```

```

28     int index = node->children.size();
29     int letter = index;
30     int letter_cost = costs[letter];

32     // Create the child and add it to the current node; then add the child to the queue
33     // Also increase the counter because one code word was just created
34     auto child = std::make_shared<Node>(letter, letter_cost);
35     node->add_child(child);
36     queue.push({cost + costs[0], child});
37     counter++;

38     // If there is a potential child left, add node to queue again
40     if (node->children.size() < r) {
41         int next_index = node->children.size();
42         queue.push({cost - letter_cost + costs[next_index], node});
43     }
44 }

46 // Return the root
47 return *pointer;
48 }

50 // DFS for traversing the tree and creating the code.
51 // Here code_word_vector is the code word created by adding the code letter (index)
52 // of the childs on the path from the root to the current node.
53 void dfs(const std::shared_ptr<Node> node, std::vector<int>& code_word_vector,
54         std::vector<std::string>& code_words) {
55     int childs = node->children.size();

56     // Add the current code word to list of code words
57     std::string code_word = to_string(code_word_vector);
58     code_words.push_back(code_word);

60     // Otherwise recursively visit all childs of the current node
62     for (int i = 0; i < childs; i++) {
63         code_word_vector.push_back(i);

64         std::shared_ptr<Node> child = (node->children)[i];
65         dfs(child, code_word_vector, code_words);

66         code_word_vector.pop_back();
67     }
68 }

70 }

72 // Function for creating a list of code words given the root of the tree
73 std::vector<std::string> create_words(Node& root) {
74     std::vector<std::string> code_words;

75     std::vector<int> code_word_vector; // Initially empty code word
76     dfs(std::make_shared<Node>(root), code_word_vector, code_words);

77     return code_words;
78 }

80 }

82 // Encode some positive number k with a prefix free code consisting of
83 // 0's and 1's.
84 std::string encode(int k) {
85     std::string code = "";
86     int bits = (k == 0) ? 0 : (32 - __builtin_clz(k));

87     for (int i = 0; i < bits; i++) {
88         if (k & (1 << i)) { // 1 at i-th position (right to left) of k
89             code = "10" + code;
90         } else {
91             code = "01" + code;
92         }
93     }

94     code = code + "00";
95     return code;
96 }

98 }

100 // Given the n least cost words, create prefix free code words by
101 // adding some prefix to each word based on its length
102 std::vector<std::string> create_code_words(const std::vector<std::string>& words) {
103     std::vector<std::string> code_words;

```



```

104     for (int i = 0; i < (int) words.size(); i++) {
106         // Get the current code words and its length
107         std::string code_word = words[i];
108         int length = code_word.size();

110         // Create the prefix based on length
111         std::string prefix = encode(length);
112         std::string new_code_word = prefix + code_word;

114         // Add the new prefix free code word
115         code_words.push_back(new_code_word);
116     }

118     return code_words;
}

```

8.5. Approximationsverfahren III

Das dritte Approximationsverfahren verwendet ebenfalls ein struct *Node*, welches Knoten des Baums darstellt, aus welchem am Ende die n initialen Codewörter abgelesen werden sollen:

```

1 // Node.h
2 // Node in the tree used to generate least cost words not containing
3 // (0...0) and not ending in 0.
4 struct Node {
5     // The amount of consecutive zeros when going from node to root (including this node)
6     int counter;
7
8     // Index of the letter this node has
9     int letter;
10
11     // The cost of the code letter
12     int cost;
13
14     // Pointers to child nodes
15     std::vector<std::shared_ptr<Node>> children; // Pointers to child nodes
16
17     Node(int counter, int letter, int cost);
18
19     // Add a child to the childrens
20     void add_child(std::shared_ptr<Node> child);
21
22     // Comparison for Nodes
23     bool operator<(const Node& other) const;
24 };
25
26 // Node.cpp
27 // Implementation of Node.h
28 Node::Node(int counter, int letter, int cost)
29     : counter(counter), letter(letter), cost(cost), children() {}
30
31 // Add a child to the childrens
32 void Node::add_child(const std::shared_ptr<Node> child) {
33     children.push_back(child);
34 }
35
36 // Comparison for Nodes
37 bool Node::operator<(const Node& other) const {
38     return cost > other.cost;
39 }

```

Der Hauptteil des dritten Approximationsverfahrens sieht wie folgt aus:

```

// Create tree from which the words are being generated
2 Node root = create_tree(costs, K, r, n);

4 // Get all words from the created tree
5 std::vector<std::string> words = create_words(root);
6
7 // Create an prefix free code using the code words

```

```
8 std::vector<std::string> code_words = create_code_words(K, words);
```

Die dabei benutzten Funktionen sind wie folgt implementiert:

```
// Create the n least cost code words not containing the terminal sequence 0...0 (K times)
2 Node create_tree(const std::vector<int>& costs, int K, int r, int n) {
    // For each Node in the queue we know that it has some potential child such that the results costs
    // are that of the integer in the pair.
    // That child will always be at index node.children.size.
    6
    Node root(0, -1, 0);
    8 auto pointer = std::make_shared<Node>(root);

    10 std::priority_queue<std::pair<int, std::shared_ptr<Node>>,
        std::vector<std::pair<int, std::shared_ptr<Node>>>,
    12 std::greater<std::pair<int, std::shared_ptr<Node>>>> queue;

    14 queue.push({costs[0], pointer});

    16 int counter = 1;

    18 while (counter < n) {
        // Pop the least cost node from the queue
    20 int cost = queue.top().first;
        std::shared_ptr<Node> node = queue.top().second;
    22 queue.pop();

    24 // Get the index of the next child of node and its letter cost
        int index = node->children.size();
    26 int letter = index;
        int letter_cost = costs[letter];
    28

        // Create a child note, setting counter of zeros corresponding to letter and node (parent)
    30 auto child = std::make_shared<Node>((letter == 0) ? (node->counter + 1) : 0, letter, letter_cost);

        // Make sure we will never have K consecutive zeros
    32 if (letter == 0 && node->counter == K-1) {
    34     node->add_child(nullptr);
        } else {
    36     // Otherwise add the child to the children of node and add it to the queue
        node->add_child(child);
    38     queue.push({cost + costs[0], child});
        }

    40

        // Dont count as a word if it ends in zero
    42 if (letter > 0) {
        counter++;
    44     }

    46 // If there is a potential child left, add node to queue again
        if (node->children.size() < r) {
    48     int next_index = node->children.size();
        queue.push({cost - letter_cost + costs[next_index], node});
    50     }
    52 }

    54 return *pointer;
}

56 // DFS for traversing the tree and creating the code.
// Here code_word_vector is the code word created by adding the code letter (index)
58 // of the childs on the path from the root to the current node.
void dfs(const std::shared_ptr<Node> node, std::vector<int>& code_word_vector,
    60 std::vector<std::string>& words) {
    // Dont consider nodes being null (which are the node where we would have K consecutive zeros)
    62 if (node == nullptr) {
        return;
    64     }

    66 int childs = node->children.size();

    68 // Only add the code word if it does not end in 0
    if (node->counter == 0) {
    70     std::string code_word = to_string(code_word_vector);
        words.push_back(code_word);
    }
```

```

72     }

74     // Recursively visit all childs of the current node
75     for (int i = 0; i < childs; i++) {
76         code_word_vector.push_back(i);

78         std::shared_ptr<Node> child = (node->children)[i];
79         dfs(child, code_word_vector, words);

80         code_word_vector.pop_back();

82     }
83 }

84 // Function for creating a list of code words given the root of the tree
85 std::vector<std::string> create_words(Node& root) {
86     std::vector<std::string> code_words;

88     std::vector<int> code_word_vector; // Initially empty code word
89     dfs(std::make_shared<Node>(root), code_word_vector, code_words);

92     return code_words;
93 }

94 // Create code words using the n least cost words (not containing terminal sequence 0...0, K zeros;
95 // and not ending in 0) by adding the terminal sequence to their end
96 std::vector<std::string> create_code_words(int K, std::vector<std::string>& words) {
97     std::vector<std::string> code_words;
98     std::string suffix = "";

100     for (int i = 0; i < K; i++) {
101         suffix += "0";
102     }

104     for (int i = 0; i < (int) words.size(); i++) {
105         std::string word = words[i];
106         code_words.push_back(word + suffix);
107     }

108     return code_words;
109 }

```

8.6. ILP-Formulierung

Zuletzt soll der Quellcode der ILP-Loesung angegeben werden. Der Hauptteil dieses Verfahrens sieht wie folgt aus:

```

1 // Create the values a_j, where a_j is the amount of code words having cost j
2 std::vector<int> vector_a = solve_ilp(m, letters, costs);
3
4 if (vector_a.size() == 0) {
5     return {};
6 }
7
8 // Construct the code words from the vector a
9 std::vector<std::string> code_words = construct_code_words(vector_a, costs);

```

Das eigentliche ILP, sowie die Konstruktion der Codewoerter aus den a_j sind wie folgt implementiert:

```

1 // The ILP Program
2 // Returns {_, a_1, a_2, ..., a_m} where a_j is the amount of code words of cost j
3 // Note that m is the maximum code word cost and everything will be 1-index.
4 std::vector<int> solve_ilp(int m, const std::vector<std::pair<int, char32_t>>& letters,
5     const std::vector<int>& costs) {
6     // amount of code letters
7     int r = costs.size();

8     // Amount of letters (ei. amount of needed codewords)
9     int n = letters.size();

10
11     // Create a new ilp solver using SCIP

```

```

13     operations_research::MPSolver solver("Karp-ILP",
14         operations_research::MPSolver::SCIP_MIXED_INTEGER_PROGRAMMING);
15
16     // Setup a time limit and the tolerance gap
17     solver.set_time_limit(60 * 1000); // 60 seconds
18     solver.SetSolverSpecificParametersAsString("limits/gap=0"); // 0 percent
19
20     // 1. Define variables
21     // 1.1 y_{ij}, e.i., if the i-th letter has a code word with cost j (binary)
22     std::vector<std::vector<operations_research::MPVariable*>> vector_y(n+1,
23         std::vector<operations_research::MPVariable*>(m+1));
24
25     for (int i = 1; i <= n; i++) {
26         for (int j = 1; j <= m; j++) {
27             vector_y[i][j] = solver.MakeIntVar(0.0, 1.0, "y-" + std::to_string(i) + "-" + std::to_string(j));
28         }
29     }
30
31     // 1.2 b_j. e.i., amount of real prefixes of cost j
32     std::vector<operations_research::MPVariable*> vector_b(m+1);
33
34     for (int j = 1; j <= m; j++) {
35         vector_b[j] = solver.MakeIntVar(0.0, solver.infinity(), "b-" + std::to_string(j));
36     }
37     // ---
38
39     // 2. Constraints
40     // 2.1 Make sure that each letter has exactly one code words
41     for (int i = 1; i <= n; i++) {
42         operations_research::MPConstraint* constraint = solver.MakeRowConstraint(1, 1);
43         for (int j = 1; j <= m; j++) {
44             constraint->SetCoefficient(vector_y[i][j], 1.0);
45         }
46     }
47
48     // 2.2 Karp's inequality: For each j: sum_{i=1}^n y_{ij} + b_j <= sum_{k=1}^r b_{j-c_k}
49     for (int j = 1; j <= m; j++) {
50         // Count the occurrences of j-costs[k-1] over all k
51         std::vector<int> counter(m+1, 0);
52
53         for (int k = 1; k <= r; k++) {
54             if (j-costs[k-1] < 0) {
55                 continue;
56             }
57             counter[j-costs[k-1]]++;
58         }
59
60         // By definition b_0 = 1, hence RHS is counter[0]
61         int RHS = counter[0];
62         operations_research::MPConstraint* constraint
63             = solver.MakeRowConstraint(-solver.infinity(), RHS);
64
65         // Add the y_{ij} for (i=1,...,n) to LHS
66         for (int i = 1; i <= n; i++) {
67             constraint->SetCoefficient(vector_y[i][j], 1.0);
68         }
69
70         // Subtract counter[k] * b_k from LHS for k=1,...,m
71         for (int k = 1; k <= m; k++) {
72             if (counter[k] > 0) {
73                 constraint->SetCoefficient(vector_b[k], -counter[k]);
74             }
75         }
76
77         // Add b_j to LHS
78         constraint->SetCoefficient(vector_b[j], 1.0);
79     }
80     // ---
81     // 3. Objective
82     operations_research::MPObjective* objective = solver.MutableObjective();
83     // Set the objective to the total cost
84     for (int i = 1; i <= n; i++) {
85         for (int j = 1; j <= m; j++) {
86             objective->SetCoefficient(vector_y[i][j], letters[i-1].first * j);
87         }
88     }
89 }

```

```

89     objective->SetMinimization();
    // ---

91

    // Solve the integer linear program
93     operations_research::MPSolver::ResultStatus result_status = solver.Solve();

95     // Print basic results
    if (result_status == operations_research::MPSolver::OPTIMAL) {
97         std::cout << "Status:_" << result_status << std::endl;
        std::cout << "Optimal_solution_found!:" << std::endl;
99         std::cout << "Min_value=" << objective->Value() << std::endl;

101         // Construct a_j using y_{ij}
        std::vector<int> vector_a(m+1);
103         for (int j = 1; j <= m; j++) {
            int sum = 0;
105             for (int i = 1; i <= n; i++) {
                sum += vector_y[i][j]->solution_value();
107             }
            vector_a[j] = sum;
109         }
        return vector_a;
111     } else {
        std::cout << "No_optimal_solution_found:_" << result_status << std::endl;
113         return {};
    }
115 }

    // Function constructing the code words given the vector a_j, where a_j is the amount
117 // of code words having cost j, 1 <= j <= m
    std::vector<std::string> construct_code_words(const std::vector<int>& vector_a,
119         const std::vector<int>& costs) {
        int m = vector_a.size()-1; // maximum code word cost
121         int r = costs.size(); // amount of code letters

123         // Remaining code words to create (we still have to create h[j] code words of cost j)
        std::vector<int> vector_h = vector_a;

125

        // Keep track of the amount of non-zero elements of h;
127         // Stop creating new code words if counter is 0
        int counter = 0;
129         for (int j = 1; j <= m; j++) {
            if (vector_h[j] != 0) {
131                 counter++;
            }
133         }
        // Actual set of code words
135         std::vector<std::string> code_words;

137         // Queue will store the next code words to add
        std::queue<std::string> queue;
139         queue.push("");

141         while (counter > 0) {
            // Pop current word and its costs from the queue
143             std::string string = queue.front();
            queue.pop();
145             int cost = get_costs(string, costs);

147             // If h[cost] > 0 (assuming 1 <= cost <= m), add this word to the code words
            if (cost <= m && vector_h[cost] > 0) {
149                 code_words.push_back(string);
                vector_h[cost]--;

151                 if (vector_h[cost] == 0) {
153                     counter--;
                }
155             } else {
                // Otherwise push the r words with prefix string to the queue
157                 for (int k = 1; k <= r; k++) {
                    queue.push(string + code_letters[k-1]);
159                 }
            }
161         }
        return code_words;
163     }

```

9. Literatur

- [1] Huffman, David A. (1952), «A Method for the Construction of Minimum-Redundancy Codes», Proceedings of the IRE, Vol. 40, Iss. 9
- [2] Golin, Mordecai J.; Mathieu, Claire; Young, Neal E. (2012), «Huffman Coding with Letter Costs: A Linear-Time Approximation Scheme», SIAM Journal on Computing, Vol. 41, Iss. 3
- [3] Karp, Richard M. (1961), «Minimum-redundancy coding for the discrete noiseless channel», IRE Transactions on Information Theory, Vol. 7, Iss. 1